MLOps, LLMOps & AIOps Made Simple

Taking models to production, keeping them healthy, and evaluating them after deployment

MLflow · Model registry · Deployment · FastAPI · TF Serving · Docker · Kubernetes
CI/CD/CT · Testing · Monitoring & observability · Drift · A/B tests · Retraining
Orchestration · SageMaker, Azure ML, Vertex AI · LLMOps · AIOps · Responsible AI

Prepared by Rahul Kothuri

How to use this guide

This is the fourth guide in the series, after *Machine Learning*, *Deep Learning* and *RAG & AI Agents Made Simple*. The earlier guides were about **building** models. This one is about **operating** them: tracking experiments, packaging and deploying models, testing and monitoring them, detecting drift, retraining safely, running LLM applications in production, using AI for IT operations, and doing all of it securely and responsibly.

It follows the course order – MLOps foundations → deployment → containers and Kubernetes → CI/CD, testing and monitoring → drift, A/B testing and retraining → orchestration and cloud platforms → LLMOps → security, governance and responsible AI – with dedicated chapters on **post-deployment evaluation** and **AIOps**.

Each concept follows the same pattern: **plain-words idea → diagram or chart → only the formulas, commands or code that matter**. Code blocks are short sketches that show the shape of a tool, not complete programs.

Box	What it holds
The idea in plain words	the concept without jargon
Real-life analogy	something familiar to anchor it
Key equations	formulas worth knowing, every symbol explained
Worked example / When to use it	numbers and practical guidance
Watch out	common mistakes and limits

Charts marked *real* come from actual computations in Python (MLflow-style runs, a batching benchmark, INT8 quantisation, PSI and KS tests, a statistical A/B test, anomaly detection, permutation importance). Charts marked *illustrative* show an idea, not measured results.

Contents

How to use this guide **1**

1 Introduction to MLOps: Principles & Practices **3**
1.1 Why MLOps? . . . 3
1.2 DevOps vs MLOps . . . 4
1.3 Core principles . . . 4
1.4 The MLOps lifecycle . . . 4
1.5 MLOps maturity levels . . . 5
1.6 Roles and tools at a glance . . . 5

2 Experiment Tracking, Model Registry & Versioning **6**
2.1 Experiment tracking . . . 6
2.2 MLflow . . . 6
2.3 Model registry . . . 7
2.4 Versioning data and environments . . . 8

3 Model Deployment, FastAPI & TensorFlow Serving **9**
3.1 How predictions are served . . . 9
3.2 Deployment strategies . . . 9
3.3 Building ML APIs with FastAPI . . . 10
3.4 Model serving with TensorFlow Serving . . . 12
3.5 Choosing a serving approach . . . 12

4 Docker, Kubernetes & Model Optimisation **13**
4.1 Containerisation with Docker . . . 13
4.2 Kubernetes for ML workloads . . . 14
4.3 Model optimisation for production . . . 15

5 CI/CD & Automated Testing for ML **18**
5.1 CI, CD and CT . . . 18
5.2 Automated testing for ML . . . 19

6 Model Monitoring & Observability **21**
6.1 Why ML needs special monitoring . . . 21
6.2 Monitoring vs observability . . . 21
6.3 The four golden signals (service level) . . . 22
6.4 Model-level metrics . . . 22
6.5 SLIs, SLOs and error budgets . . . 22
6.6 Alerting well . . . 22

7 Drift Detection, A/B Testing & Retraining **24**
7.1 Types of drift . . . 24
7.2 Detecting drift . . . 24
7.3 A/B testing for ML models . . . 26
7.4 Model retraining strategies . . . 27

8 ML Pipeline Orchestration & Cloud MLOps **29**
8.1 Pipeline orchestration . . . 29
8.2 Cloud MLOps platforms . . . 30

9 LLMOps: Operating LLM Applications **31**
9.1 What changes with LLMs? . . . 31
9.2 Key LLMOps practices . . . 31
9.3 Cost optimisation for LLM applications . . . 32
9.4 Monitoring LLM applications . . . 33

10 Evaluation After Deployment **35**
10.1 Why evaluate after deployment? . . . 35
10.2 The ground-truth problem . . . 35
10.3 The post-deployment evaluation loop . . . 36
10.4 Online evaluation methods . . . 36
10.5 Evaluating LLM and RAG apps in production . . . 36
10.6 What "good" looks like: a scorecard . . . 37

11 AIOps: AI for IT Operations **38**
11.1 What is AIOps? . . . 38
11.2 Core capabilities . . . 38
11.3 Measuring operations . . . 39
11.4 AIOps for ML systems . . . 40

12 Security, Governance, Responsible AI & Explainability **41**
12.1 Security in MLOps . . . 41
12.2 Governance . . . 41
12.3 Responsible AI . . . 42
12.4 Model explainability . . . 43

13 MLOps Best Practices & Design Patterns **45**
13.1 Best practices checklist . . . 45
13.2 Design patterns . . . 45
13.3 Anti-patterns to avoid . . . 46

14 Cheat Sheet & Interview Revision **47**
14.1 Formula sheet . . . 47
14.2 Interview quick revision . . . 47

Further Reading **49**

CHAPTER 1

Introduction to MLOps: Principles & Practices

1.1 Why MLOps?

The idea in plain words

Training a good model in a notebook is maybe 10% of the work. **MLOps** (Machine Learning Operations) is the set of practices that gets models into production *reliably* and keeps them working: versioning, automation, testing, deployment, monitoring and retraining. It is DevOps adapted for systems whose behaviour depends on **data**, not just code.

Real-life analogy

A chef who invents a great dish (data scientist) is not the same as a restaurant serving it 500 times a night with the same quality, safe ingredients and a way to notice when customers stop liking it (MLOps).

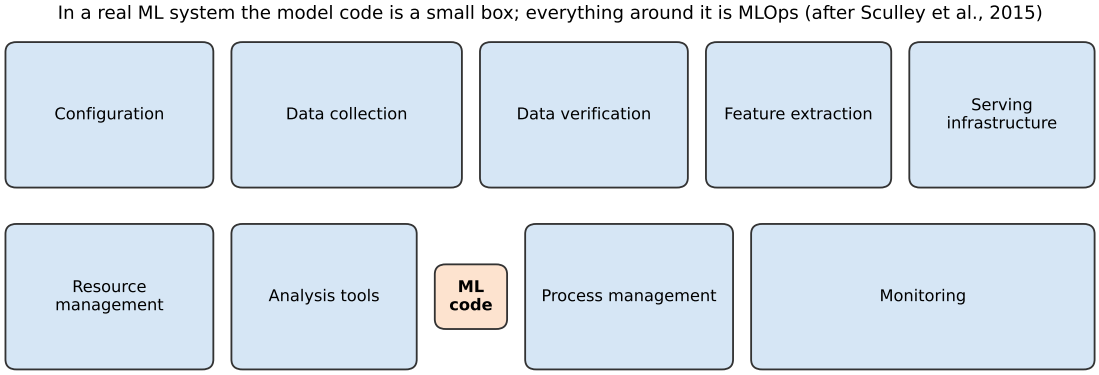


Figure 1.1: The ML code is a small part of a production ML system.

Common reasons ML projects fail to reach or survive production: "works on my laptop" environments, nobody knows which data or code produced the deployed model, manual hand-offs, no tests, no monitoring, and models that silently decay as the world changes.

1.2 DevOps vs MLOps

	DevOps (software)	MLOps (machine learning)
What is versioned	Code	Code + **data** + **model** + **parameters** + **environment**
Testing	Unit, integration tests	Also data tests, model quality, fairness, drift
Failure mode	Crash, error, bug	Also *silent* failure: runs fine, predictions get worse
Pipeline	CI + CD	CI + CD + **CT** (continuous training)
Monitoring	Uptime, latency, errors	Also data drift, prediction drift, model accuracy
Team	Dev + Ops	Data scientists, ML engineers, data engineers, Ops, business

1.3 Core principles

1. **Versioning everything** – code (Git), data (DVC, lakeFS, dataset snapshots), models (registry), environments (Docker, lock files).
2. **Reproducibility** – anyone can recreate a model exactly: same data, code, parameters, random seeds and library versions.
3. **Automation** – pipelines instead of manual steps: training, testing, deployment, retraining.
4. **Continuous X** – continuous integration, delivery, training and monitoring.
5. **Testing** – code, data and model quality, as automatic gates.
6. **Monitoring & feedback** – watch the system, data and model in production, and feed what you learn back into training.
7. **Governance** – lineage, approvals, audit trails, access control, responsible AI checks.
8. **Collaboration** – shared tools (tracking server, registry, feature store) across teams.

1.4 The MLOps lifecycle

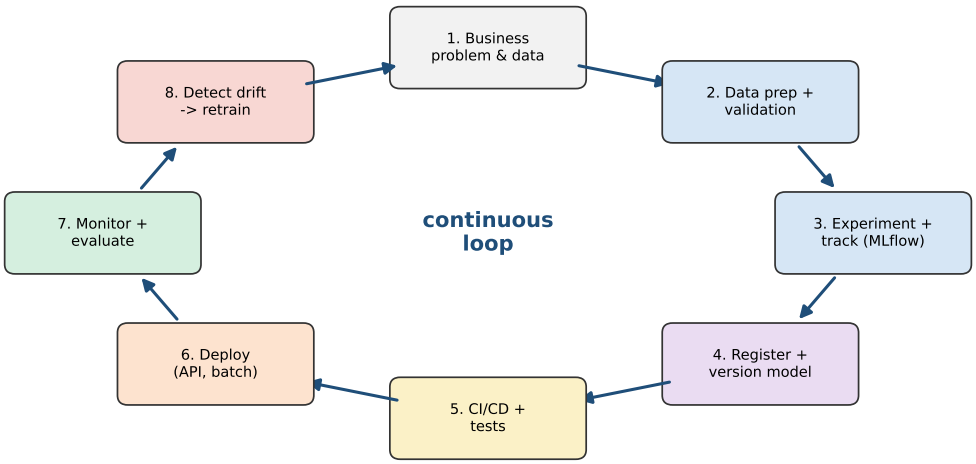


Figure 1.2: MLOps is a loop, not a straight line: monitoring feeds back into retraining.

1.5 MLOps maturity levels

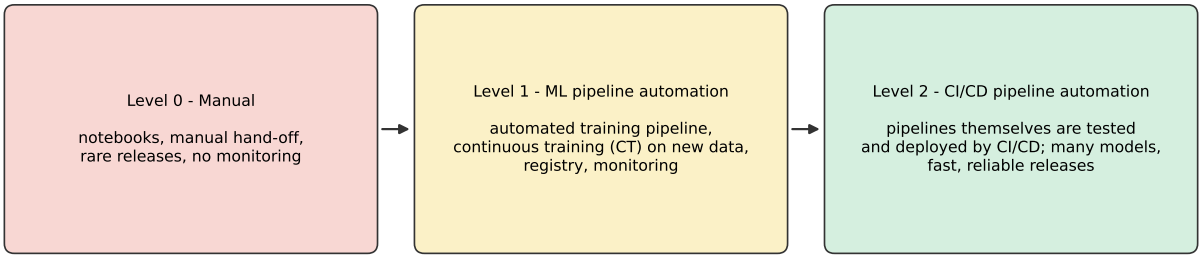


Figure 1.3: Maturity levels (based on Google's MLOps levels 0–2).

Remember

Most teams start at Level 0. The biggest single jump in value is usually: put everything in Git, track experiments, use a model registry, containerise the model, and add basic monitoring.

1.6 Roles and tools at a glance

Need	Typical tools
Experiment tracking, registry	MLflow, Weights & Biases, Neptune, cloud-native registries
Data / pipeline versioning	Git, DVC, lakeFS, Delta Lake
Serving	FastAPI, TensorFlow Serving, TorchServe, Triton, BentoML, KServe, vLLM (LLMs)
Packaging & running	Docker, Kubernetes (AKS, EKS, GKE), Helm
CI/CD	GitHub Actions, Azure DevOps, Jenkins, GitLab CI, ArgoCD
Orchestration	Airflow, Kubeflow Pipelines, Prefect, Dagster, Azure ML / SageMaker / Vertex pipelines
Monitoring	Prometheus + Grafana, Evidently, WhyLabs, Arize, Azure Monitor, CloudWatch, LangSmith (LLMs)
Feature store	Feast, Tecton, Databricks / SageMaker / Vertex feature stores

CHAPTER 2

Experiment Tracking, Model Registry & Versioning

2.1 Experiment tracking

The idea in plain words

Every training attempt is a **run**. For each run, record *what went in* (parameters, data version, code commit) and *what came out* (metrics, plots, the model file). Later you can compare runs, find the best one, and reproduce it exactly – instead of "final_v3_really_final.pkl".

Real-life analogy

A lab notebook for experiments, kept automatically and searchable.

2.2 MLflow

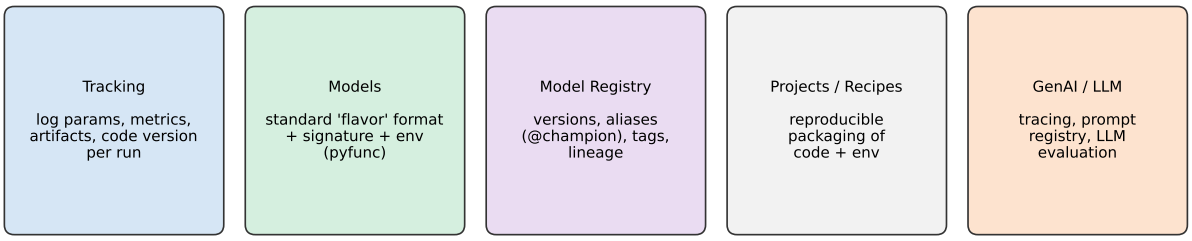


Figure 2.1: The main parts of MLflow.

Term	Meaning
Experiment	A named group of runs (e.g. "churn-model")
Run	One execution of training code
Parameters	Inputs you set: `n_estimators=200`, `max_depth=8`
Metrics	Numbers you measure; can be logged per step/epoch: accuracy, F1, RMSE
Artifacts	Files: model, plots, confusion matrix, data sample, requirements
Tags	Free labels: `git_commit`, `data_version`, `owner`
Signature	The model's expected input and output schema
Tracking server	Stores runs (backend DB) and artifacts (blob storage) for the team

```
import mlflow
mlflow.set_tracking_uri("http://mlflow-server:5000")
mlflow.set_experiment("churn-model")
with mlflow.start_run(run_name="rf-200-d8"):
    mlflow.log_params({"n_estimators": 200, "max_depth": 8})
    model = RandomForestClassifier(200, max_depth=8).fit(X_train, y_train)
    mlflow.log_metric("f1", f1_score(y_test, model.predict(X_test)))
    mlflow.sklearn.log_model(model, name="model",
```

```
                                input_example=X_train[:5])  # infers the signature
# or simply: mlflow.autolog()  -> logs params, metrics and model automatically
```

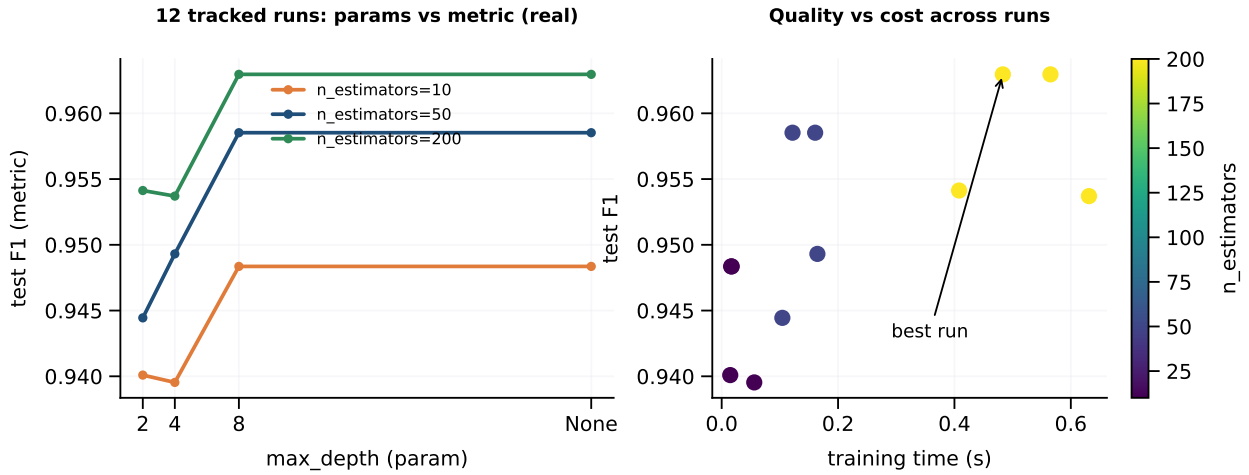


Figure 2.2: Comparing tracked runs (real): 12 random-forest runs with different parameters. The tracking UI gives the same kind of view – best F1, and what each run cost to train.

Run the UI locally with `mlflow ui`; compare runs side by side, sort by metric, and open artifacts.

2.3 Model registry

The idea in plain words

The registry is the **single source of truth for models**. Each registered model has numbered **versions**; each version links back to the run (code, data, parameters) that made it. Deployment code never points at a file path – it asks the registry for, say, "the champion version of fraud_classifier".

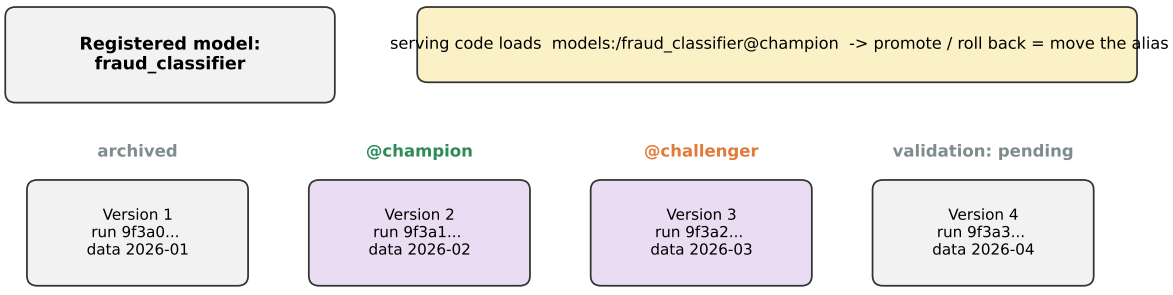


Figure 2.3: Versions, aliases and tags in a registry.

- **Versions** are immutable and auto-numbered (v1, v2, ...).
- **Aliases** are movable named pointers: `@champion`, `@challenger`. Load with `models:/fraud_classifier@champion`. Promoting or rolling back = reassigning the alias – no code change.
- **Tags** record status: `validation_status=passed`, `approved_by=...`.
- **Stages** (None / Staging / Production / Archived) are the older MLflow approach and are deprecated in favour of aliases and tags; you will still see them in older code and exams.

- **Lineage**: version → run → code commit + data version + parameters + metrics. Essential for audits and debugging.

```
mv = mlflow.register_model(f"runs:/{run_id}/model", "fraud_classifier")
client = mlflow.MlflowClient()
client.set_registered_model_alias("fraud_classifier", "challenger", mv.version)
model = mlflow.pyfunc.load_model("models:/fraud_classifier@champion")
```

Remember

A typical promotion flow: new version → automatic validation (metrics, fairness, latency) → `@challenger` → shadow or canary test → approval gate (often manual for regulated use cases) → move `@champion`. Keep the previous champion for instant rollback.

2.4 Versioning data and environments

- **Data**: you can't reproduce a model without the exact data. Use DVC (Git-like pointers to data in cloud storage), lakeFS/Delta Lake time travel, or immutable dated snapshots; log the data version or hash as a run tag.
- **Environment**: pin library versions (`requirements.txt` / lock file), log them with the model, and build Docker images from them.
- **Semantic versioning** for services: MAJOR.MINOR.PATCH (break.add.fix). Model versions and API versions are related but separate.

Watch out

Training–serving skew: preprocessing written twice (once in the notebook, once in the API) drifts apart. Package preprocessing *with* the model (a scikit-learn `Pipeline` or a custom MLflow pyfunc) or use a feature store.

CHAPTER 3

Model Deployment, FastAPI & TensorFlow Serving

3.1 How predictions are served

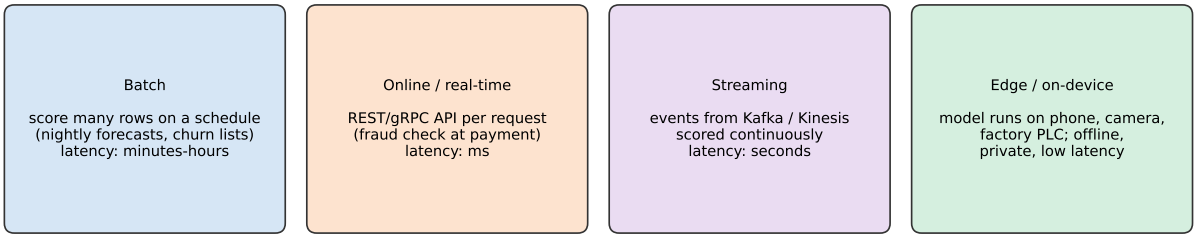


Figure 3.1: Four ways to serve predictions. Choose by how fast the answer is needed.

When to use it

Batch when results can wait (nightly forecasts, weekly churn scores) – cheapest and simplest. **Online** when a user or system waits for the answer (fraud check, recommendations). **Streaming** for continuous event flows (IoT, clickstreams). **Edge** when there's no reliable network, or for privacy and very low latency.

3.2 Deployment strategies

The idea in plain words

Never swap the old model for the new one blindly. Release in a way that limits the damage if the new model is worse, and makes rollback instant.

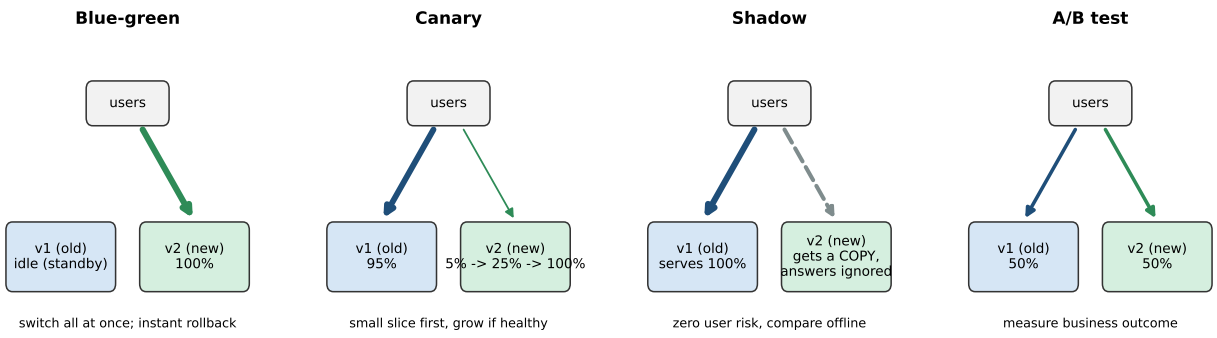


Figure 3.2: Four release strategies.

Strategy	How	Pros	Cons
Recreate	Stop old, start new	Simple	Downtime
Rolling	Replace instances a few at a time	No downtime; Kubernetes default	Old and new mixed during rollout
Blue-green	Two full environments; switch the router	Instant switch and rollback	Double infrastructure for a while
Canary	1–5% of traffic to new, then grow	Limits blast radius; real traffic	Needs good metrics and automation
Shadow	New model gets a copy of traffic; its answers are not used	Zero user risk; compare on real data	Extra compute; no business-outcome signal
A/B test	Split traffic to measure an outcome	Measures real business impact	Needs time and enough traffic
Multi-armed bandit	Shift traffic towards the winner while testing	Less regret than fixed A/B	More complex analysis

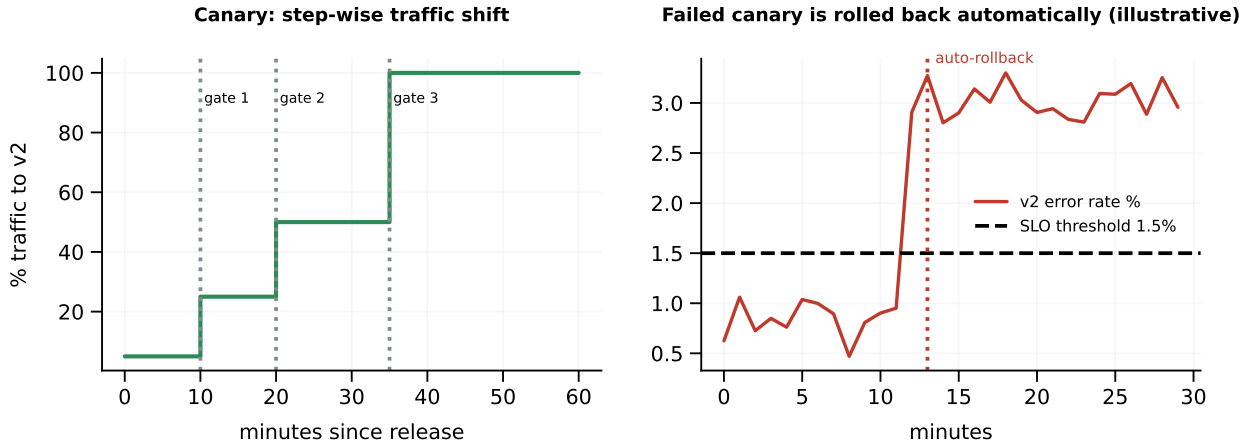


Figure 3.3: A canary moves through gates; if the error rate breaks the SLO, traffic returns to the old version.

Remember

Canary vs A/B: a canary mainly *limits risk* during rollout; an A/B test mainly *measures which version is better* for a business metric. Shadow mode is the safest first step for a brand-new model.

3.3 Building ML APIs with FastAPI

The idea in plain words

FastAPI turns a Python function into a fast web API. It uses Python type hints and **Pydantic** models to validate requests automatically, and generates interactive docs at `/docs`.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pydantic import BaseModel, Field
import mlflow

class Features(BaseModel):
    tenure_months: int = Field(ge=0)
    monthly_charges: float = Field(gt=0)
```

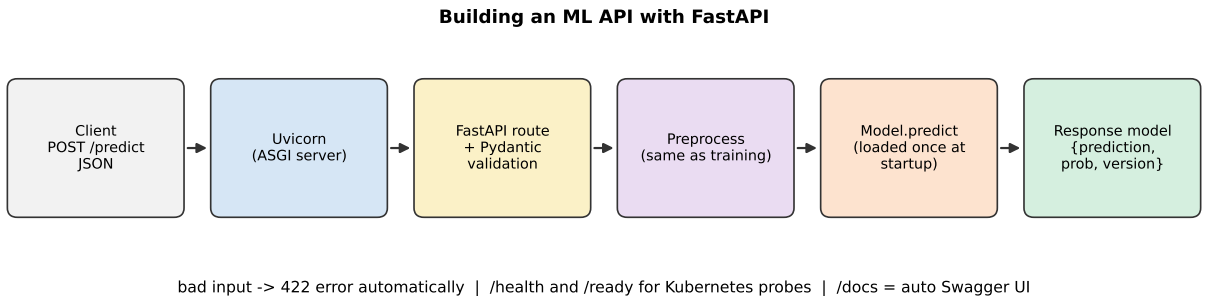


Figure 3.4: The life of a prediction request.

```
    contract: str

class Prediction(BaseModel):
    churn_probability: float
    model_version: str

@asynccontextmanager
async def lifespan(app):                      # load ONCE at startup, not per request
    app.state.model = mlflow.pyfunc.load_model("models:/churn@champion")
    yield

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health(): return {"status": "ok"}

@app.post("/predict", response_model=Prediction)
def predict(f: Features):
    p = app.state.model.predict(pd.DataFrame([f.model_dump()]))[0]
    return Prediction(churn_probability=float(p), model_version="v12")
# run:  uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

Good practices

- Load the model once at startup; keep preprocessing identical to training.
- Validate inputs strictly (types, ranges); invalid input returns HTTP 422 automatically.
- Return the **model version** with every prediction and log a request ID – you need both for monitoring and post-deployment evaluation.
- Provide `/health` (liveness) and `/ready` (model loaded) endpoints for Kubernetes.
- Use a batch endpoint for many rows; use `async` for I/O-bound work (calling other services), but CPU-heavy model inference doesn't benefit from `async` – use more workers or a model server.
- Add authentication (API key or OAuth2/JWT), rate limiting and timeouts.
- Test with `TestClient` and pytest.

3.4 Model serving with TensorFlow Serving

The idea in plain words

A dedicated, high-performance C++ server for TensorFlow models. You give it a folder of exported **SavedModel** versions; it serves them over REST and gRPC, batches requests, and hot-swaps to new versions without downtime – no web code needed.

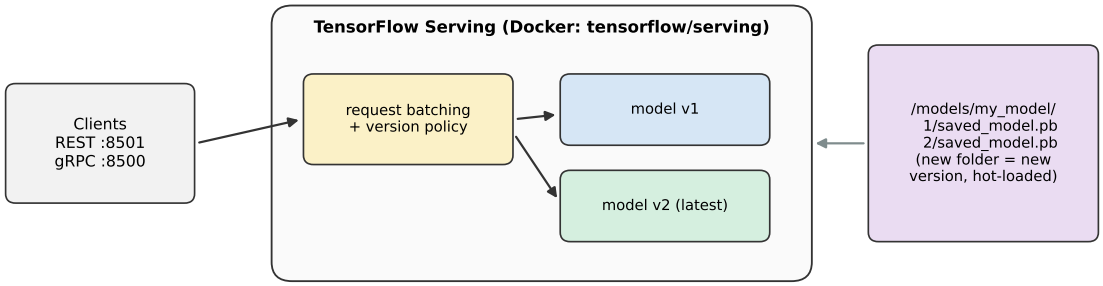


Figure 3.5: TensorFlow Serving loads versioned SavedModels from a folder.

```
model.export("models/my_model/2")              # Keras 3; or tf.saved_model.save(model, path)
docker run -p 8501:8501 -p 8500:8500 \
  -v "$PWD/models/my_model:/models/my_model" -e MODEL_NAME=my_model tensorflow/serving

curl -X POST http://localhost:8501/v1/models/my_model:predict \
     -d '{"instances": [[5.1, 3.5, 1.4, 0.2]]}'
```

- **REST** on port 8501 (easy, JSON); **gRPC** on 8500 (faster, binary, typed).
- **Version policy**: serve latest (default), all, or specific versions; a config file can label versions (e.g. `stable`, `canary`).
- **Batching**: `-enable_batching` groups requests for GPU efficiency.
- Inspect a model's signature with `saved_model_cli show -dir ... -all`.

3.5 Choosing a serving approach

Option	Best for
FastAPI (+ Uvicorn)	Any Python model, custom business logic, quick to build
TensorFlow Serving	TensorFlow/Keras models at scale, versioning built in
TorchServe	PyTorch models
NVIDIA Triton	Many frameworks on GPUs, dynamic batching, ensembles
BentoML, MLflow serve	Package any model as a service quickly
KServe / Seldon	Model serving on Kubernetes with autoscaling, canary, explainers
vLLM, TGI	High-throughput LLM serving (continuous batching, paged attention)
Managed endpoints	SageMaker, Azure ML online endpoints, Vertex AI endpoints

CHAPTER 4

Docker, Kubernetes & Model Optimisation

4.1 Containerisation with Docker

The idea in plain words

A **container** packages your app with everything it needs – Python version, libraries, model file, code – so it runs the same on your laptop, a test server and the cloud. No more "it works on my machine".

Real-life analogy

A shipping container: whatever is inside, every ship, crane and truck knows how to handle the box.

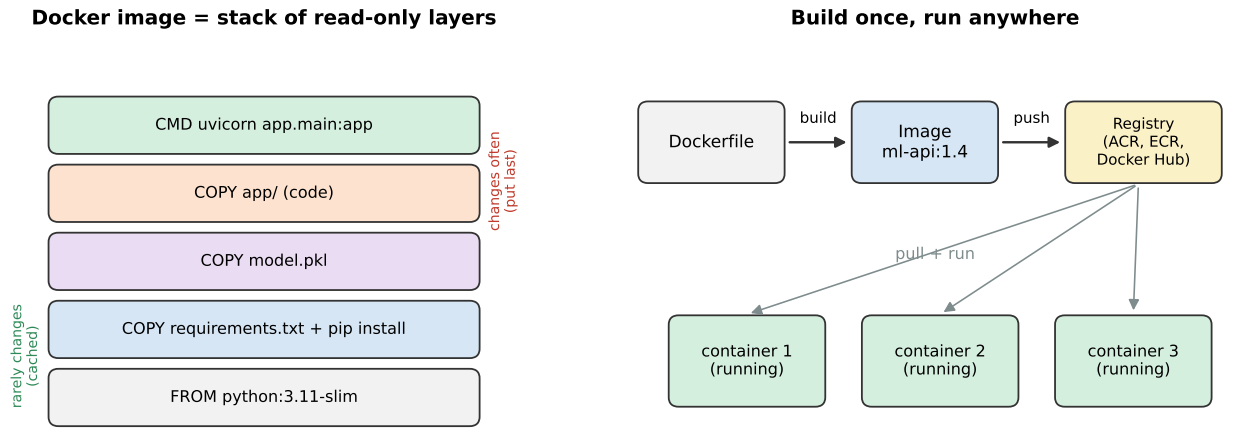


Figure 4.1: An image is a read-only template built in layers; a container is a running instance of an image.

Term	Meaning
Dockerfile	Recipe of instructions to build an image
Image	Packaged, read-only template (layers), tagged e.g. `ml-api:1.4`
Container	A running instance of an image (isolated process)
Registry	Stores images: Docker Hub, Azure ACR, AWS ECR, Google Artifact Registry
Volume	Persistent storage outside the container's life
Docker Compose	Run several containers together (API + Redis + DB) from one YAML file

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt      # cached unless requirements change
COPY model/ ./model/
COPY app/ ./app/
RUN useradd -m appuser
USER appuser                                               # don't run as root
```

```
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

docker build -t ml-api:1.4 .     docker run -p 8000:8000 ml-api:1.4
```

Best practices: small base images (`-slim`), copy dependency files before code (layer caching), **multi-stage builds** (build tools in stage 1, only runtime in stage 2), `.dockerignore` (no data or `.venv`), pin versions, never bake secrets into images (use environment variables or a secret store), run as non-root, scan images for vulnerabilities, and tag images with a version or Git SHA – not just `latest`.

4.2 Kubernetes for ML workloads

The idea in plain words

Docker runs one container. **Kubernetes (K8s)** runs *many* containers across many machines: it keeps the requested number running, restarts crashed ones, spreads load, scales up and down, and rolls out new versions without downtime.

Real-life analogy

Docker is a shipping container; Kubernetes is the port that decides which ship each container goes on, replaces lost ones, and adds ships when it gets busy.

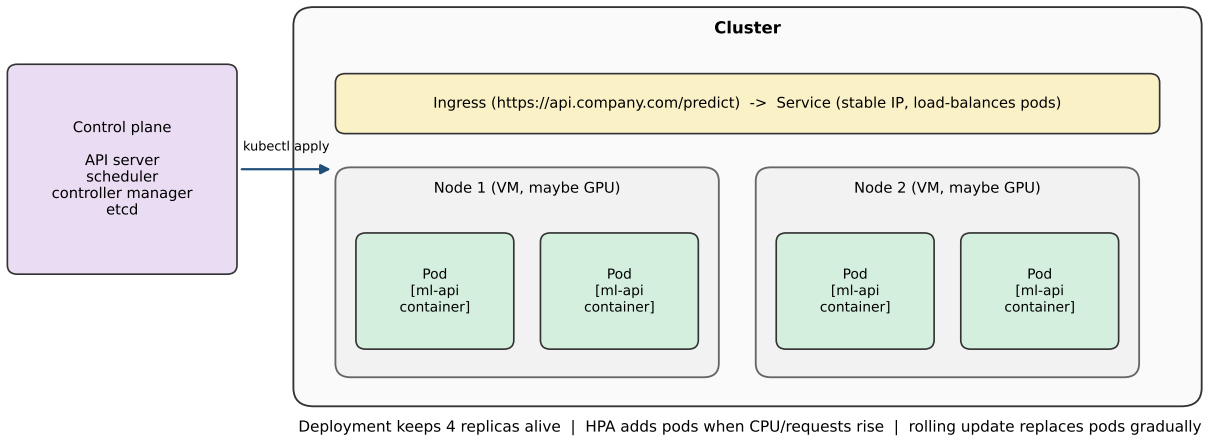


Figure 4.2: Main Kubernetes building blocks.

Object	Purpose
Pod	Smallest unit: one (or a few) containers sharing network and storage
Deployment	Desired state: "run 4 replicas of image ml-api:1.4"; handles rolling updates and rollback
Service	Stable address and load balancing across pods
Ingress	HTTP routing from outside (domain, path, TLS)
ConfigMap / Secret	Configuration and credentials injected into pods
HPA	Horizontal Pod Autoscaler: changes the number of replicas based on CPU, memory or custom metrics
Job / CronJob	Run-to-completion work: training, batch scoring, scheduled retraining
Namespace	Separate environments or teams (dev, staging, prod)
Liveness / readiness probes	Restart unhealthy pods; send traffic only to ready ones
Resource requests/limits	CPU, memory and `nvidia.com/gpu` per pod

Key equations

HPA rule: $\text{desired replicas} = \left\lceil \text{current replicas} \times \dfrac{\text{current metric}}{\text{target metric}} \right\rceil$

e.g. 4 pods at 90% CPU with target 60%: $\lceil 4 \times 90/60 \rceil = 6$ pods.

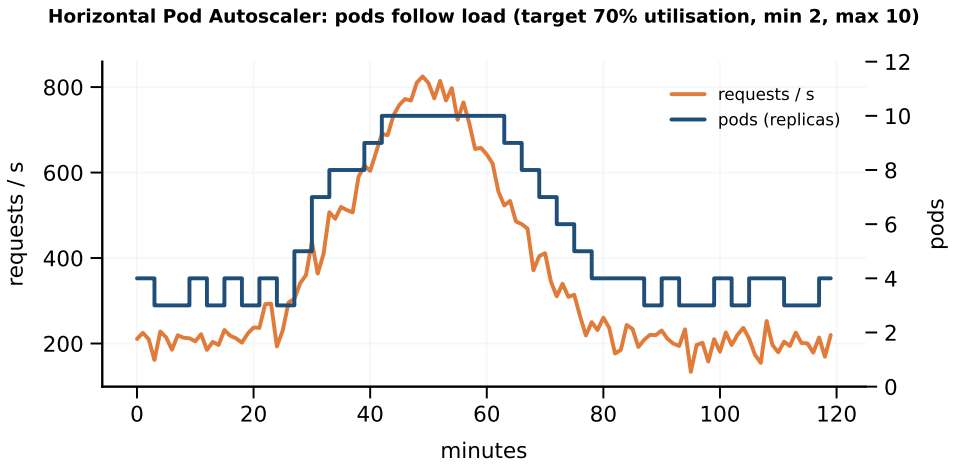


Figure 4.3: Autoscaling follows demand (simulated).

ML-specific: GPU node pools with node selectors/taints; large model images and slow start-up (use readiness probes and keep warm replicas); Kubeflow for ML pipelines and notebooks; KServe for model serving with scale-to-zero and canaries; Helm charts to package all the YAML; managed clusters: AKS (Azure), EKS (AWS), GKE (Google).

4.3 Model optimisation for production

The idea in plain words

Production cares about **latency**, **throughput**, **memory** and **cost**, not only accuracy. Optimisation makes a model smaller and faster, ideally with little or no loss of accuracy.

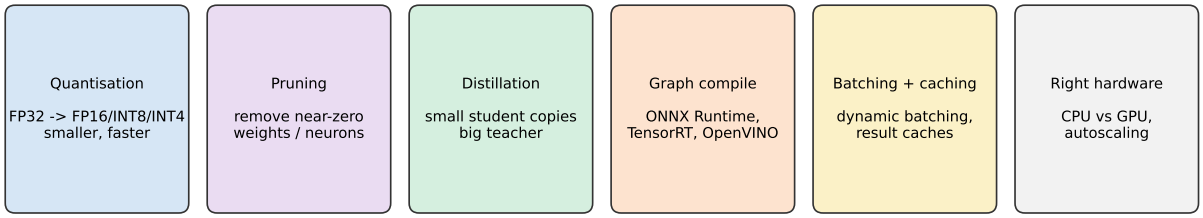


Figure 4.4: The main optimisation levers.

Key equations

Latency = time for one request (report p50, p95, p99). **Throughput** = requests handled per second.

Little's law: concurrent requests in the system = throughput × average latency
e.g. 200 req/s at 0.15 s each ⇒ about 30 requests in flight ⇒ size workers and pods for that.

Quantisation (symmetric INT8): $q = \text{round}\left(\frac{w}{s}\right), \quad s = \frac{\max|w|}{127}, \quad \hat{w} = q \cdot s$

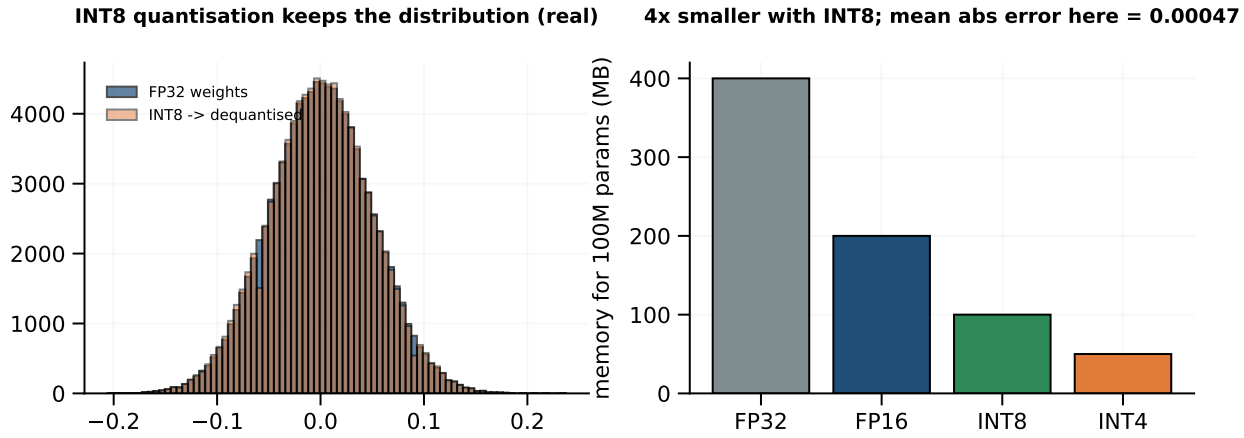


Figure 4.5: Real INT8 quantisation of 100,000 weights: the distribution is preserved, memory falls 4× vs FP32.

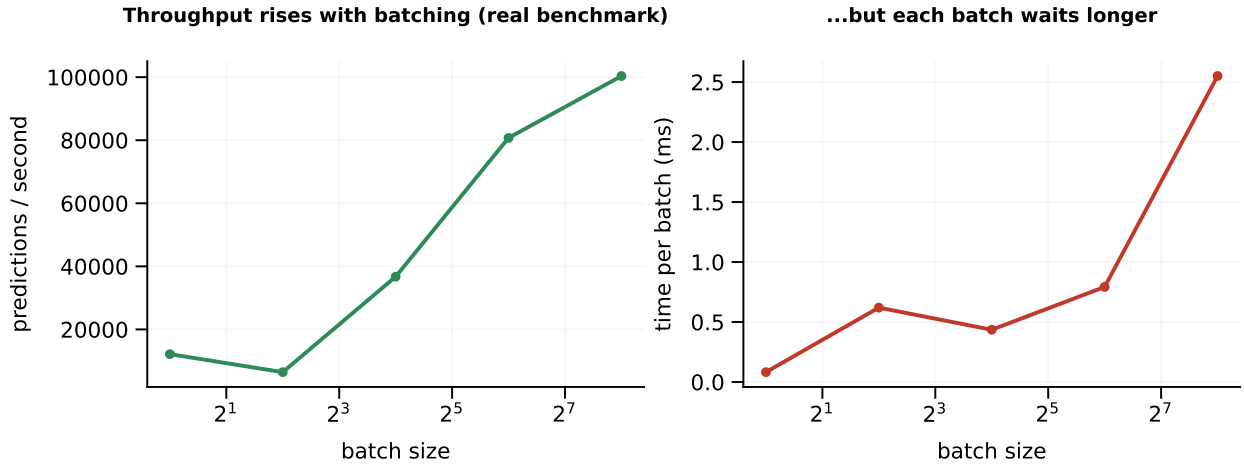


Figure 4.6: Real benchmark of a small neural network: batching raises throughput, but each batch takes longer. Dynamic batching waits a few milliseconds to collect requests.

Technique	What it does	Typical effect
Quantisation	Lower-precision numbers (FP16, INT8, INT4)	2–4× smaller, often faster; small accuracy loss
Pruning	Remove small weights/neurons/heads	Smaller; speed-up needs structured pruning or sparse kernels
Knowledge distillation	Train a small student on a big teacher's outputs	Much smaller model, most of the accuracy
ONNX export + runtime	Framework-neutral graph, optimised kernels	Faster CPU/GPU inference, portable
TensorRT / OpenVINO	Hardware-specific compilers (NVIDIA / Intel)	Large GPU/CPU speed-ups
Batching	Process several requests together	Much higher throughput
Caching	Reuse results for repeated inputs	Near-zero latency for hits
Feature pre-computation	Compute heavy features ahead of time	Lower request latency

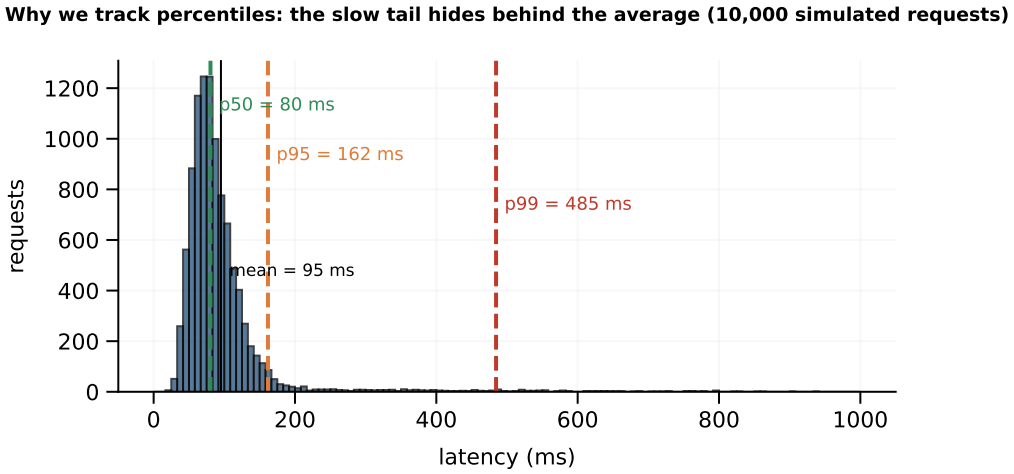


Figure 4.7: The average hides the slow tail – SLOs are written on p95/p99.

Watch out

Always re-evaluate accuracy *after* optimising, on the same test set, and benchmark on the real target hardware.

CHAPTER 5

CI/CD & Automated Testing for ML

5.1 CI, CD and CT

The idea in plain words

CI (continuous integration): every code change is automatically built and tested. **CD** (continuous delivery/deployment): tested changes are automatically packaged and released to staging and production. **CT** (continuous training): unique to ML – new data or drift automatically triggers retraining, evaluation and (if better) promotion.

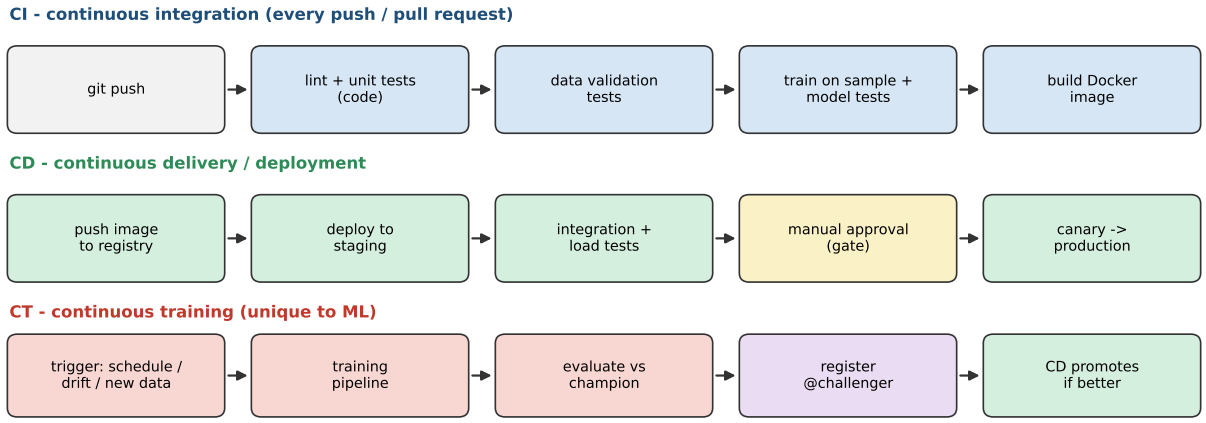


Figure 5.1: CI, CD and CT pipelines for an ML system.

Remember

Continuous delivery = always *ready* to release, with a final manual approval. **Continuous deployment** = released automatically when all checks pass. Regulated ML (credit, health) usually keeps a human approval gate.

```
# .github/workflows/ml.yml  (GitHub Actions sketch)
on: [push, pull_request]
jobs:
  test-train-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/                        # unit + data + model tests
      - run: python train.py --sample             # quick training on a sample
      - run: python evaluate.py --min-f1 0.85     # fail the build below threshold
      - run: docker build -t $REGISTRY/ml-api:${{ github.sha }} .
      - run: docker push $REGISTRY/ml-api:${{ github.sha }}
  deploy-staging:
    needs: test-train-build
    environment: staging                          # protected env -> approval rules
    steps: [ { run: "kubectl set image deployment/ml-api api=$REGISTRY/ml-api:${{ github.sha }}" } ]
```

Equivalent pipelines can be written in Azure DevOps, GitLab CI or Jenkins; **GitOps** tools such as ArgoCD deploy whatever is declared in a Git repository.

5.2 Automated testing for ML

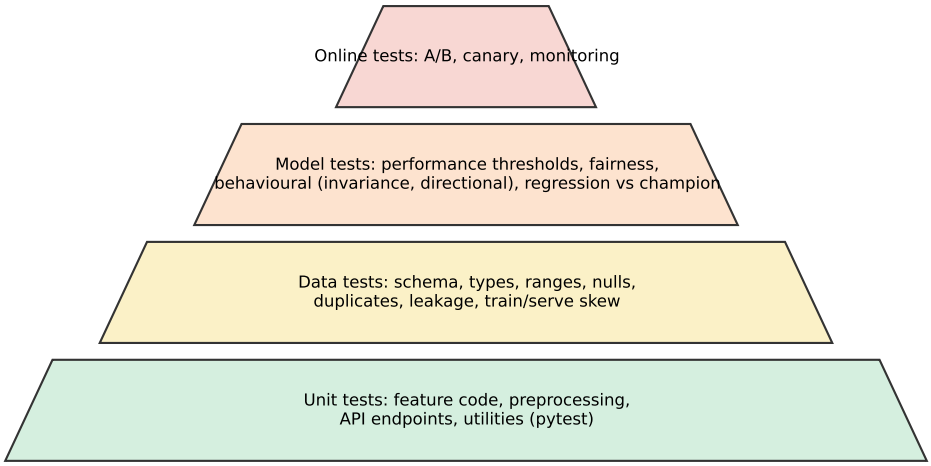


Figure 5.2: Test layers for an ML system.

5.2.1 Code tests

Unit tests for feature functions, preprocessing and utilities; API tests (status codes, response schema, invalid input → 422); test the model-loading path. Use pytest, fixtures and small synthetic data.

5.2.2 Data tests

- **Schema**: expected columns and types (a field changed from string to float is a classic silent failure).
- **Validity**: ranges (age 0–120), allowed categories, null share, uniqueness of IDs, duplicates.
- **Freshness and volume**: is today's data present, and is the row count normal?
- **Leakage**: no target-derived features; no overlap between train and test.
- **Train/serve skew**: the same row gives the same features offline and online.
- Tools: Great Expectations, Pandera, Evidently, TFDV.

5.2.3 Model tests

- **Performance thresholds**: F1/AUC/RMSE above a minimum on a fixed test set, *and* not worse than the current champion.
- **Slice tests**: performance on important segments (region, device, new customers) – averages hide weak groups.
- **Behavioural tests**:
  - *Minimum functionality*: simple cases must be right ("terrible service" → negative).
  - *Invariance*: irrelevant changes don't change the output (changing a customer's name).
  - *Directional expectation*: a known change moves the output the right way (higher income → approval probability shouldn't fall).
- **Fairness**: gaps between groups within limits (Chapter 12).

- **Robustness**: missing values, extreme values, noise don't crash or wildly change predictions.
- **Non-functional**: latency and memory budgets, model file size, reproducibility (same seed ⇒ same metrics within tolerance).

```
def test_model_beats_threshold(model, test_data):
    X, y = test_data
    assert f1_score(y, model.predict(X)) >= 0.85

def test_invariance_to_name(model, sample_row):
    a, b = sample_row.copy(), sample_row.copy(); b["customer_name"] = "Someone Else"
    assert abs(model.predict_proba(a)[0,1] - model.predict_proba(b)[0,1]) < 1e-6

def test_latency(model, sample_row):
    t0 = time.perf_counter(); model.predict(sample_row)
    assert time.perf_counter() - t0 < 0.05            # 50 ms budget
```

5.2.4 Integration, load and online tests

Staging deployment with end-to-end calls; load tests (Locust, k6) to check p95 latency at expected traffic; then shadow, canary and A/B tests in production.

CHAPTER 6

Model Monitoring & Observability

6.1 Why ML needs special monitoring

The idea in plain words

Normal software fails loudly – errors, crashes, timeouts. ML models usually fail **silently**: the API keeps returning `200 OK` while predictions slowly get worse because the world has changed. Monitoring must therefore watch the *data* and the *predictions*, not just the servers.

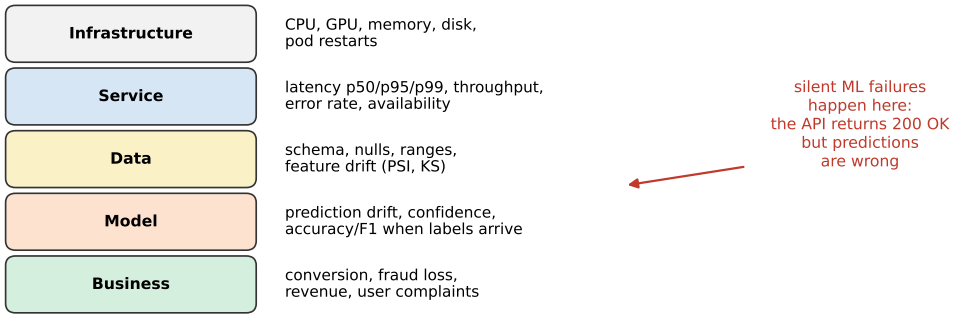


Figure 6.1: Five layers to monitor, from servers to business results.

6.2 Monitoring vs observability

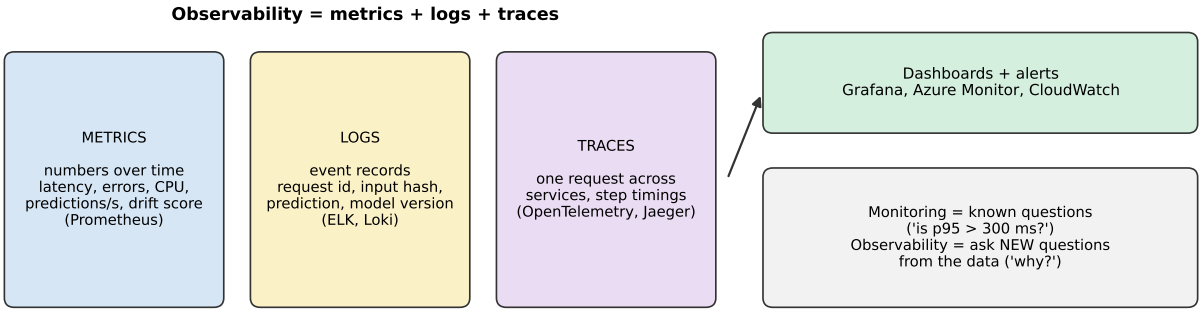


Figure 6.2: The three pillars of observability.

- **Metrics**: numeric time series, cheap to store, great for dashboards and alerts.
- **Logs**: detailed events. For ML, log the request ID, timestamp, model version, (hashed or sampled) inputs, prediction and confidence – you need these later to join with true labels.
- **Traces**: follow one request across services (API → feature store → model → database) with timings for each step. **OpenTelemetry** is the open standard for all three.

6.3 The four golden signals (service level)

Signal	Meaning	ML example
Latency	How long requests take (p50/p95/p99)	p95 prediction time 120 ms
Traffic	Demand on the system	350 predictions/s
Errors	Rate of failed requests	0.2% HTTP 5xx, 1% validation errors
Saturation	How "full" the system is	GPU 92% busy, queue growing

6.4 Model-level metrics

- **Data quality**: missing values, out-of-range values, new categories, schema changes.
- **Data drift**: feature distributions vs training (Chapter 7).
- **Prediction drift**: distribution of outputs (share of "fraud", average score, confidence).
- **Model performance**: accuracy, F1, AUC, RMSE – once ground-truth labels arrive (often delayed).
- **Business KPIs**: fraud losses, conversions, manual-review workload.
- **Fairness over time**: metric gaps between groups.

6.5 SLIs, SLOs and error budgets

Key equations

SLI (indicator): a measured value, e.g. share of requests answered in < 300 ms.
SLO (objective): a target for the SLI, e.g. 99.9% of requests successful over 30 days.
SLA (agreement): a contract with consequences if missed.

Error budget $= 1 - \text{SLO}$. For 99.9% and 10 million requests a month: $0.001 \times 10^7 = 10{,}000$ failed requests allowed.

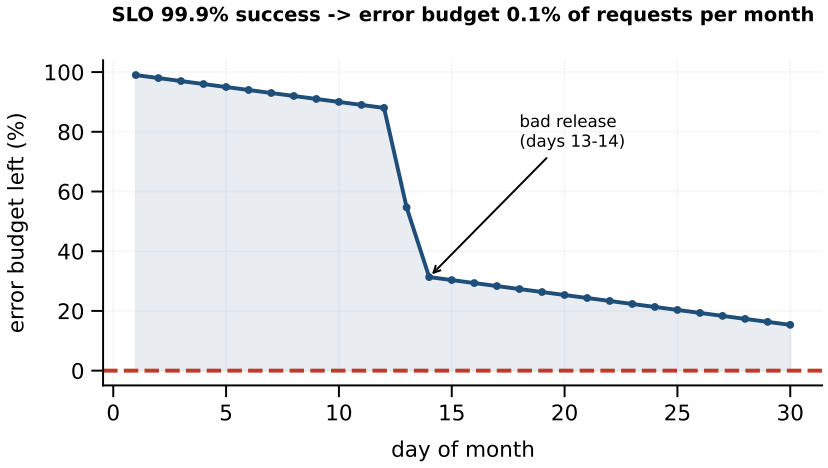


Figure 6.3: When the error budget runs low, slow down risky releases and fix reliability first.

6.6 Alerting well

- Alert on **symptoms users feel** (SLO burn, error spikes, drift on key features), not on every metric.

- Use sensible windows and thresholds to avoid **alert fatigue**; group related alerts (AIOps, Chapter 11).
- Every alert needs an owner and a **runbook** (what to check, how to roll back).
- Tools: Prometheus + Alertmanager + Grafana; Azure Monitor + Application Insights; AWS CloudWatch; Google Cloud Monitoring; Evidently / WhyLabs / Arize for ML-specific checks.

CHAPTER 7

Drift Detection, A/B Testing & Retraining

7.1 Types of drift

The idea in plain words

A model learns patterns from training data. In production the world keeps moving: customers change, prices change, sensors age, fraudsters adapt. **Drift** is any change that makes production different from what the model learned.

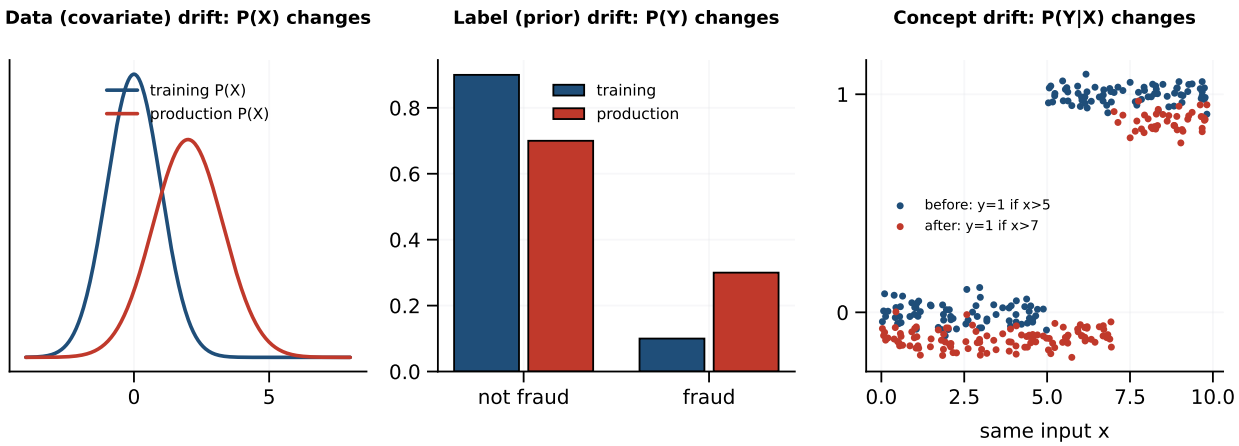


Figure 7.1: Data drift, label drift and concept drift.

Type	What changes	Example
Data / covariate / feature drift	$P(X)$	More young users after a marketing campaign
Label / prior drift	$P(Y)$	Fraud rate jumps from 1% to 3%
Concept drift	$P(Y \mid X)$	Same income now means higher default risk after a recession
Prediction drift	$P(\hat{Y})$	Model suddenly predicts "high risk" twice as often
Data-quality drift	pipeline / schema	A column switches unit, type or starts arriving empty

Concept drift can be **sudden** (new regulation), **gradual**, **incremental** or **recurring/seasonal** (festive shopping). **Model drift** (model decay) is the resulting drop in performance.

Remember

Data drift does not always hurt the model, and concept drift can hurt it with *no* data drift at all. Drift statistics are early-warning **proxies**; the final check is performance once labels arrive.

7.2 Detecting drift

Compare a **reference** window (training or a healthy production period) with a **current** window (last day or week), feature by feature.

Key equations

Population Stability Index: $\mathrm{PSI} = \sum_{i=1}^{B} (a_i - e_i) \ln \dfrac{a_i}{e_i}$

e_i = share of reference rows in bin i, a_i = share of current rows (bins = deciles of the reference). Rule of thumb: < 0.1 no significant change, 0.1–0.25 moderate, > 0.25 significant.

Kolmogorov–Smirnov (numeric): $D = \max_x |F_{\mathrm{ref}}(x) - F_{\mathrm{cur}}(x)|$, drift if p-value < 0.05.

Chi-square (categorical): $\chi^2 = \sum \dfrac{(O-E)^2}{E}$ over category counts.

Jensen–Shannon divergence: $\mathrm{JS}(P\|Q) = \frac{1}{2}\mathrm{KL}(P\|M) + \frac{1}{2}\mathrm{KL}(Q\|M)$, $M = \frac{1}{2}(P+Q)$; bounded, symmetric.

Wasserstein distance: "how much mass must move, how far" – in the feature's own units.

Worked example

One bin holds 10% of training rows and 20% of today's rows: contribution $= (0.20 - 0.10)\ln(0.20/0.10) = 0.1 \times 0.693 = 0.069$. Add up all ten bins to get the PSI.

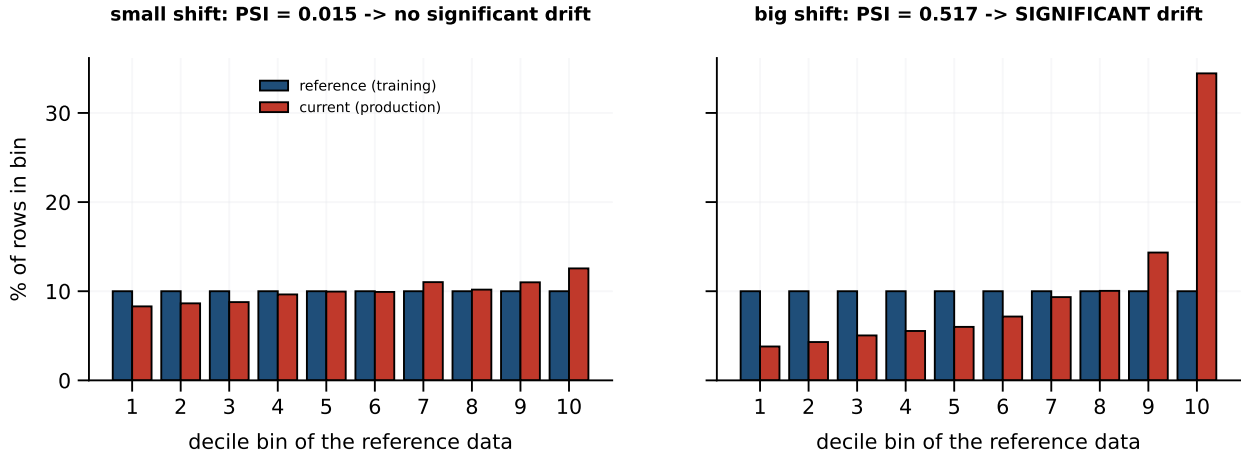


Figure 7.2: Real PSI computation on 5,000 rows: a 1-point mean shift gives PSI 0.015; a larger shift and wider spread give PSI above 0.25.

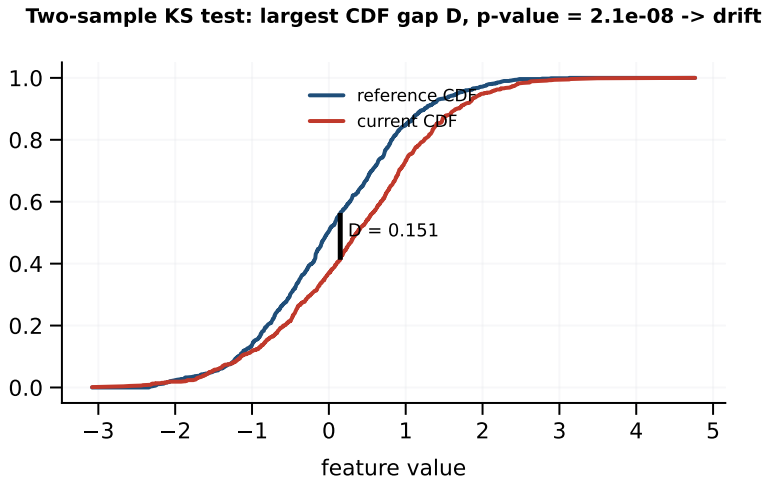


Figure 7.3: Real two-sample KS test: the statistic is the biggest vertical gap between the two cumulative distributions.

- **Large data warning**: with huge samples, p-value tests (KS, chi-square) flag tiny, harmless differences. Distance metrics with thresholds (PSI, Wasserstein, JS) are often more

practical. Evidently, for example, defaults to KS/chi-square for small reference data and Wasserstein/Jensen–Shannon for larger data.

- **Dataset-level drift**: e.g. alert if more than 50% of features drift, or if any *important* feature drifts.
- **Embeddings / text**: drift on embedding distances, or train a "domain classifier" to tell reference from current – an AUC well above 0.5 means drift.
- **Concept drift with labels**: track rolling performance; detectors like DDM, ADWIN and Page–Hinkley watch the error rate for change points.

7.3 A/B testing for ML models

The idea in plain words

Randomly split live users: group A gets the current model, group B the new one. After enough traffic, use statistics to decide whether B really improves the **business metric** (conversion, revenue, resolution rate) or whether the difference is just noise.

Key equations

Two-proportion z-test: $z = \dfrac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$, $\hat{p}$ = pooled rate; significant at 5% if $|z| > 1.96$.

Sample size per group (approx.): $n \approx \dfrac{(z_{1-\alpha/2} + z_{1-\beta})^2 \left[p_1(1-p_1) + p_2(1-p_2)\right]}{(p_2 - p_1)^2}$, $z = 1.96$ (95%), 0.84 (80% power).

Worked example

A: 1,000 of 10,000 converted (10.0%). B: 1,090 of 10,000 (10.9%). Pooled $\hat{p} = 0.1045$; SE $= \sqrt{0.1045 \times 0.8955 \times (2/10000)} = 0.00433$; $z = 0.009/0.00433 = 2.08$; p $\approx 0.037 \Rightarrow$ B is better at the 5% level.

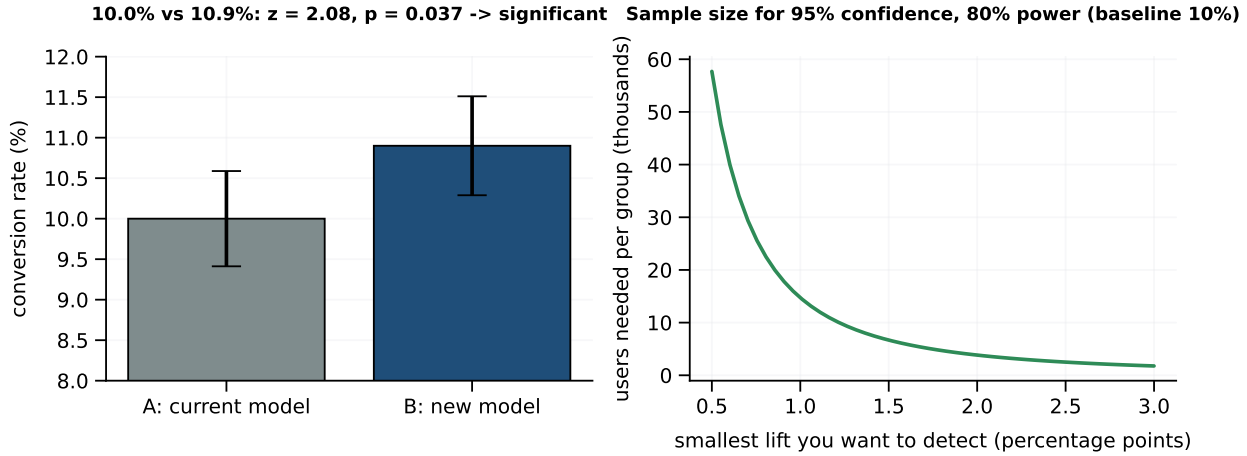


Figure 7.4: Left: the worked example (real statistics, 95% confidence intervals). Right: detecting small lifts needs far more users.

Watch out

Decide the metric, sample size and duration **before** starting; don't stop early when it "looks significant" (peeking); randomise by user, not by request; run full weekly cycles; watch guardrail metrics (latency, complaints) too. Bandits (Thompson sampling) shift traffic to the winner during the test, reducing lost value but making inference harder.

7.4 Model retraining strategies

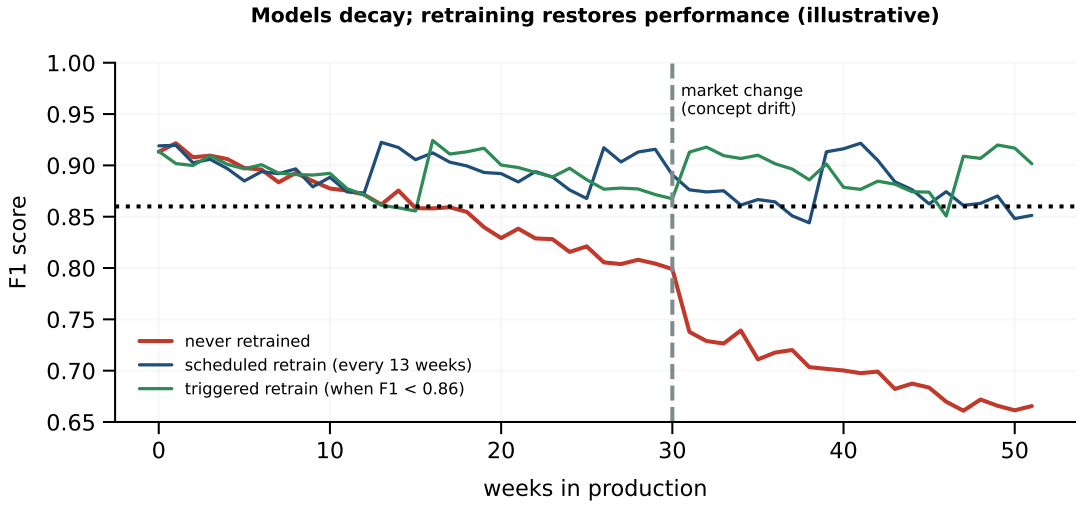


Figure 7.5: Without retraining, performance decays; scheduled and triggered retraining restore it (illustrative).

Strategy	How	Good for
Manual / ad hoc	Someone notices and retrains	Early stage only
Scheduled (periodic)	Every day/week/month	Predictable change; simple; may waste or lag
Trigger-based	Drift score, performance drop, or N new labels	Efficient; needs reliable monitoring
Continuous / online	Update with each new batch or stream	Fast-changing data (ads, recommendations)

Which data? Full retrain on all history; a **sliding window** (last N months – forgets old patterns); **weighted** (recent data counts more); or **incremental / fine-tuning** from the previous model (cheap, but risk of drift or forgetting).

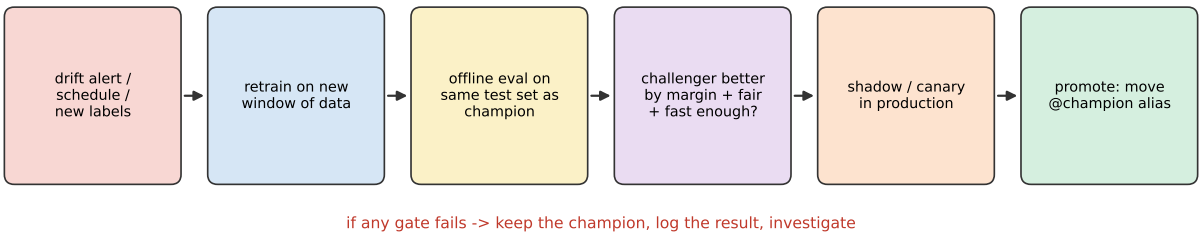


Figure 7.6: Champion–challenger: a retrained model must beat the current one before it is promoted.

Watch out

Automatic retraining is not automatic deployment. Always validate the challenger against the champion on the same held-out data and slices, check fairness and latency, and keep a rollback path. Watch for **feedback loops**: if the model's own decisions shape future labels (e.g. only approved loans get repayment labels), retraining can reinforce its biases.

CHAPTER 8

ML Pipeline Orchestration & Cloud MLOps

8.1 Pipeline orchestration

The idea in plain words

An ML workflow has many steps – ingest, validate, build features, train, evaluate, register, deploy. An **orchestrator** runs them in the right order, in parallel where possible, on a schedule or trigger, retries failures, passes outputs between steps, and records what happened.

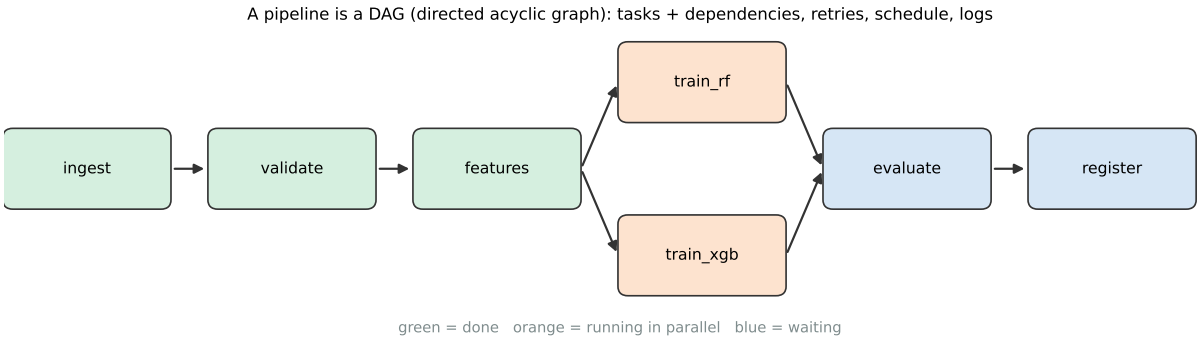


Figure 8.1: A training pipeline as a DAG.

Orchestrator	Notes
Apache Airflow	Most widely used general workflow scheduler; Python DAGs; great for data pipelines
Kubeflow Pipelines	Kubernetes-native ML pipelines; each step is a container
Prefect, Dagster	Modern Python-first orchestrators; Dagster focuses on data assets
MLflow Projects / Recipes	Lightweight reproducible runs
Cloud pipelines	SageMaker Pipelines, Azure ML pipelines, Vertex AI Pipelines (Kubeflow/TFX-based)

Good pipeline steps are idempotent (re-running gives the same result), parameterised (dates, data version), cached (skip unchanged steps), and log their inputs/outputs to the tracking server.

8.1.1 Feature stores

A feature store computes features once and serves the *same* definitions to training (offline store, history with point-in-time correctness) and inference (online store, low-latency look-ups) – removing training/serving skew and enabling reuse. Examples: Feast, Tecton, Databricks, SageMaker and Vertex AI feature stores.

8.2 Cloud MLOps platforms

The idea in plain words

The big clouds bundle the whole lifecycle – notebooks, managed training compute, experiment tracking, registries, pipelines, endpoints and monitoring – so you don't have to run the infrastructure yourself.

Capability	AWS SageMaker	Azure Machine Learning	Google Vertex AI
Workspace / IDE	SageMaker Studio	Azure ML studio, workspace	Vertex AI Workbench
Training	Training jobs, spot instances, HyperPod	Command/sweep jobs on compute clusters	Custom training jobs, hyperparameter tuning
AutoML	SageMaker Autopilot (Canvas)	Automated ML	Vertex AutoML
Tracking	SageMaker Experiments, managed MLflow	MLflow-compatible tracking built in	Vertex AI Experiments
Registry	SageMaker Model Registry (approval status)	Azure ML registry (MLflow models)	Vertex AI Model Registry
Pipelines	SageMaker Pipelines	Azure ML pipelines (+ Azure DevOps / GitHub)	Vertex AI Pipelines
Serving	Real-time, serverless, async, batch transform	Managed online endpoints, batch endpoints	Online / batch prediction endpoints
Monitoring	SageMaker Model Monitor, Clarify, CloudWatch	Model monitoring, Azure Monitor, App Insights	Vertex AI Model Monitoring
Feature store	SageMaker Feature Store	Managed feature store	Vertex AI Feature Store
GenAI	Amazon Bedrock	Azure AI Foundry / Azure OpenAI	Gemini models in Vertex AI

Remember

The concepts are the same everywhere – workspace, compute, data assets, experiments, registry, pipelines, endpoints, monitoring. Learn them once and map the product names. Product names change often; check the current documentation.

8.2.1 Typical cloud MLOps architecture

Source code in Git → CI pipeline (tests, build) → training pipeline on managed compute reading versioned data from cloud storage → metrics in tracking → register model → validation gates (drift, performance, fairness) → approval by owners → deploy to staging then production endpoints → monitor with the cloud's monitoring service → drift alerts trigger the training pipeline again.

8.2.2 Choosing

- Follow where your data and company cloud already are (security, networking, cost).
- Managed services reduce ops work but add lock-in; open tools (MLflow, Kubernetes, Airflow) keep you portable – many teams mix both.
- Watch cost: idle endpoints, oversized GPU instances, and forgotten notebooks are common waste. Use autoscaling, scale-to-zero, spot instances for training, and budgets/alerts.

CHAPTER 9

LLMOps: Operating LLM Applications

9.1 What changes with LLMs?

The idea in plain words

LLMOps is MLOps for applications built on large language models – chatbots, RAG assistants, agents. You usually don't train the model; you **choose** it, and your "model" becomes the combination of **prompts, retrieval, tools and model settings**. Outputs are free text, quality is hard to measure, behaviour can change when the provider updates the model, and cost is charged **per token**.

	Classic MLOps	LLMOps
Main artefact	Trained model weights	Prompt + model choice + RAG index + tools + parameters
Versioning	Code, data, model	Also prompts, system instructions, embedding model, index, eval sets
Evaluation	Accuracy, F1, RMSE vs labels	LLM-as-judge, human review, faithfulness, task success, safety
Failure modes	Drift, decay	Hallucination, prompt injection, toxic output, refusal, provider changes
Cost driver	Servers / GPUs	Tokens in + out, model size, number of calls (agents)
Latency	ms per prediction	Seconds; time-to-first-token and streaming matter
Customisation	Retrain	Prompting, RAG, fine-tuning (LoRA), model routing

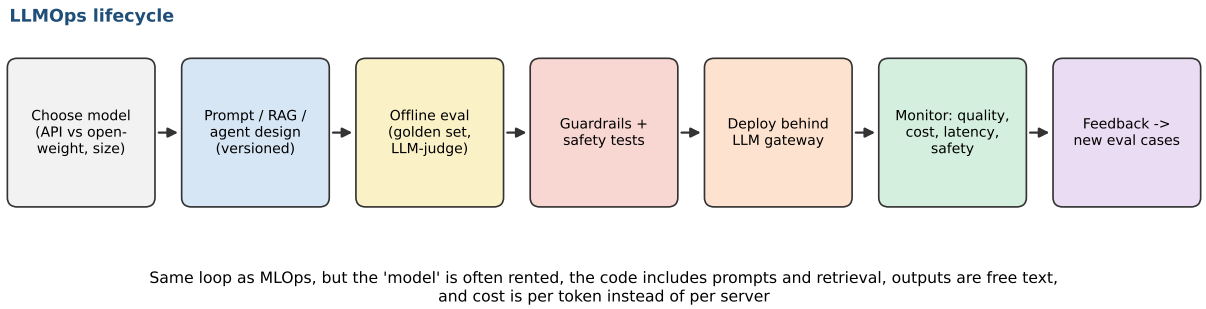


Figure 9.1: The LLMOps lifecycle.

9.2 Key LLMOps practices

1. **Prompt management**: store prompts as versioned artefacts (Git, MLflow prompt registry, LangSmith hub); review changes like code; log the prompt version with every call.
2. **Evaluation before release**: a golden dataset of real questions with expected answers; automatic scoring (exact match, LLM-as-judge rubrics, faithfulness for RAG); regression tests

in CI for every prompt, model or index change.

3. **Pin model versions**: use dated model versions where offered; re-run evals before switching.
4. **Guardrails**: input checks (prompt injection, PII, off-topic), output checks (JSON schema, grounding, toxicity), least-privilege tools, human approval for risky actions.
5. **LLM gateway**: one central proxy for keys, quotas, routing, caching, fallbacks, logging and cost.
6. **Observability**: trace every step (retrieval, tool calls, LLM calls) with tokens, latency and cost (LangSmith, MLflow tracing, Langfuse, OpenTelemetry GenAI conventions).
7. **Serving open-weight models**: vLLM or TGI with continuous batching and paged KV-cache; quantised weights (INT8/INT4); GPU autoscaling.

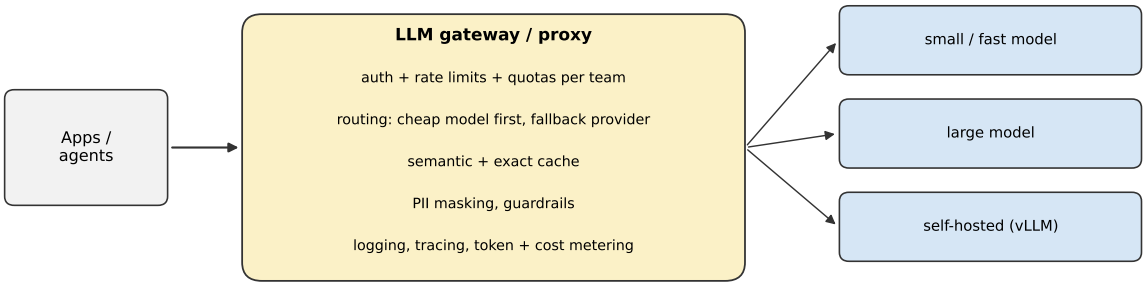


Figure 9.2: An LLM gateway centralises control.

9.3 Cost optimisation for LLM applications

Key equations

Cost per request $= \dfrac{T_{\text{in}} \times P_{\text{in}} + T_{\text{out}} \times P_{\text{out}}}{10^6}$ (T = tokens, P = price per million tokens)

Monthly cost = cost per request × requests per month × LLM calls per request (agents make several).

Worked example

Illustrative prices: large model \$3 per 1M input tokens and \$15 per 1M output tokens. A RAG call with 1,500 input and 300 output tokens costs $(1500\times3+300\times15)/10^6 = \$0.009$. At 1 million requests a month that's about \$9,000. A small model at \$0.15/\$0.60 costs $(225+180)/10^6 = \$0.000405$ per call – about \$405 a month.

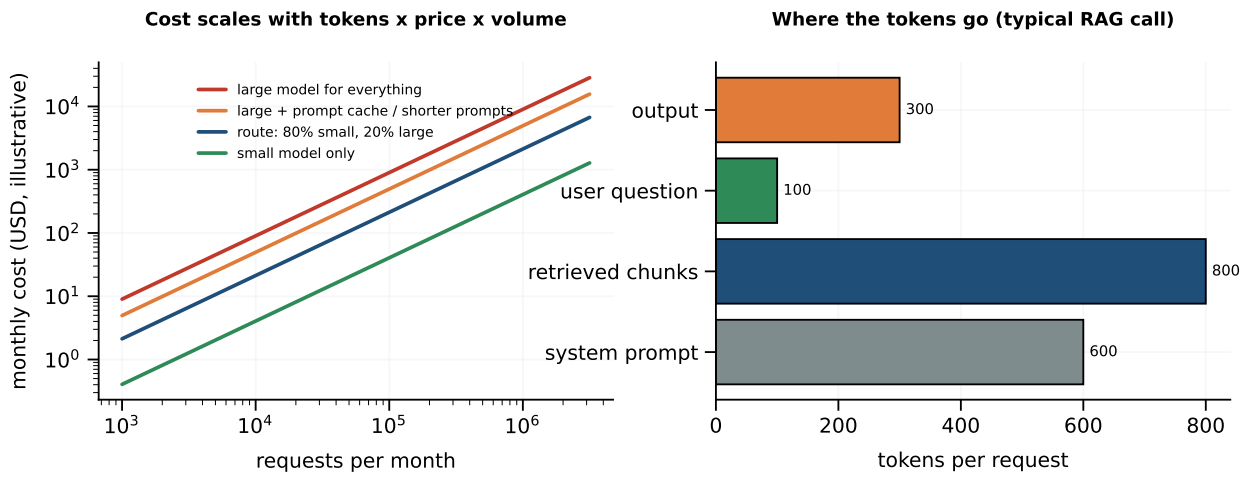


Figure 9.3: Left: cost grows linearly with volume; routing and caching move the whole line down (illustrative prices). Right: in RAG, input tokens (system prompt + retrieved chunks) usually dominate.

Lever	How it saves
Right-size the model	Use the smallest model that passes your evals; reserve large models for hard cases
Model routing / cascades	Classifier or cheap model first; escalate only when needed
Shorter prompts	Trim instructions and few-shot examples; send fewer, re-ranked chunks
Prompt (prefix) caching	Providers discount repeated prompt prefixes such as long system prompts
Response caching	Exact or semantic cache for repeated questions
Limit output	`max_tokens`, concise formats, structured output
Batch APIs	Non-urgent jobs at discounted batch rates
Fewer agent steps	Step limits; deterministic code for simple logic
Self-host when volume is high	Quantised open-weight models on vLLM; compare total cost incl. GPUs and staff
Budgets & quotas	Per-team limits and alerts in the gateway

9.4 Monitoring LLM applications

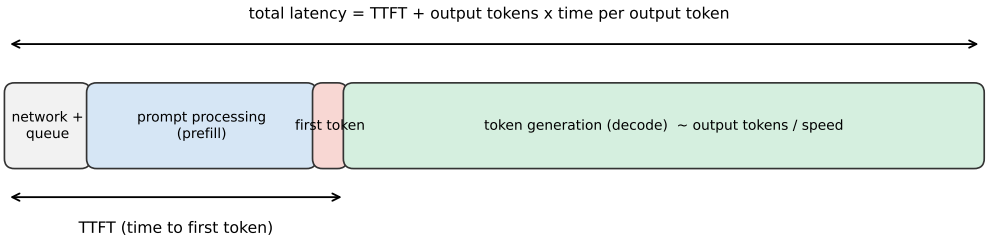


Figure 9.4: Anatomy of LLM latency.

Key equations

Total latency $\approx$ TTFT + output tokens $\times$ TPOT (TPOT = time per output token; tokens/s = 1/TPOT)

e.g. TTFT 0.6 s, 300 tokens at 50 tokens/s $\Rightarrow 0.6 + 6 = 6.6$ s; streaming makes it *feel* like 0.6 s.

Category	What to track
Performance	TTFT, total latency p50/p95, tokens/s, timeouts, rate-limit (429) errors
Cost	Tokens in/out per request, cost per request/user/feature/team, cache hit rate
Quality	Sampled LLM-judge scores (correctness, faithfulness, relevance), thumbs up/down, regeneration rate, escalation to humans
RAG	Retrieval hit rate, empty retrievals, "I don't know" rate, citation coverage
Agents	Steps per task, tool errors, loops, task success rate
Safety	Guardrail triggers, prompt-injection attempts, PII detections, refusals
Drift	Topic mix of user queries (embedding clusters), new languages, longer prompts

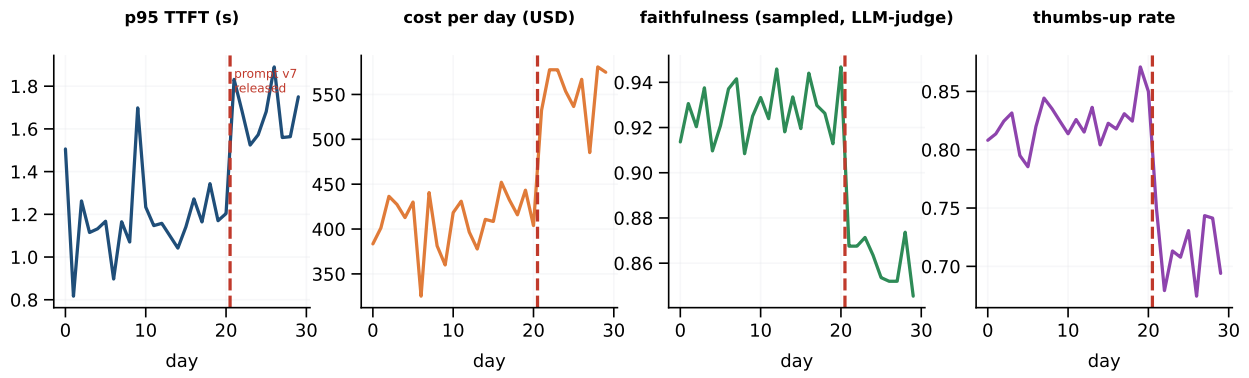


Figure 9.5: Illustrative dashboard: a bad prompt release (day 21) shows up in latency, cost and quality at once – roll back the prompt version.

CHAPTER 10

Evaluation After Deployment

10.1 Why evaluate after deployment?

The idea in plain words

Offline test scores are a promise; production is the proof. Real users send inputs you never saw, the data shifts, and what matters is the **business outcome**. Post-deployment evaluation keeps checking – continuously – that the model is still correct, fair, fast, safe and worth its cost.

	Offline evaluation (before)	Online evaluation (after)
Data	Fixed held-out test set	Live traffic, new segments
Labels	Available	Often delayed, partial or missing
Metrics	Accuracy, F1, AUC, RMSE	Same, plus business KPIs, latency, cost, feedback
Question	"Is it good enough to ship?"	"Is it still good, for everyone, in reality?"

10.2 The ground-truth problem

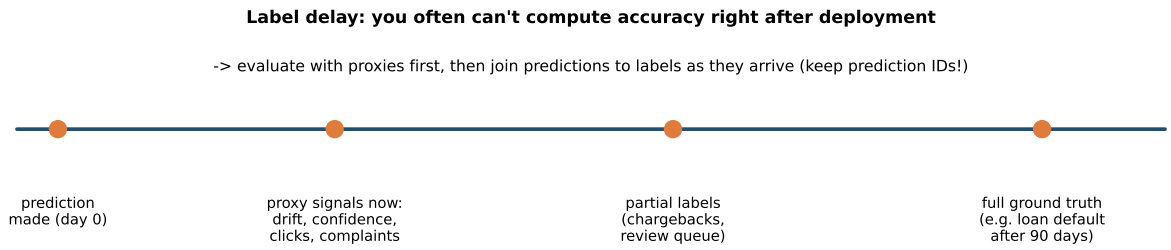


Figure 10.1: Labels often arrive days or months after the prediction.

Strategies when labels are slow or missing:

- **Proxy metrics** right away: input and prediction drift, confidence distribution, rate of "uncertain" predictions, downstream behaviour (clicks, overrides by staff, complaints).
- **Label joins**: log a prediction ID; when outcomes arrive (chargeback, churn, repayment), join them back and compute the real metric for that week's predictions.
- **Sampling + human review**: label a random sample (plus low-confidence cases) through a review queue.
- **Estimating performance without labels**: methods such as confidence-based performance estimation (e.g. CBPE) predict accuracy from calibrated probabilities – useful, but only if the model is well calibrated and there is no concept drift.

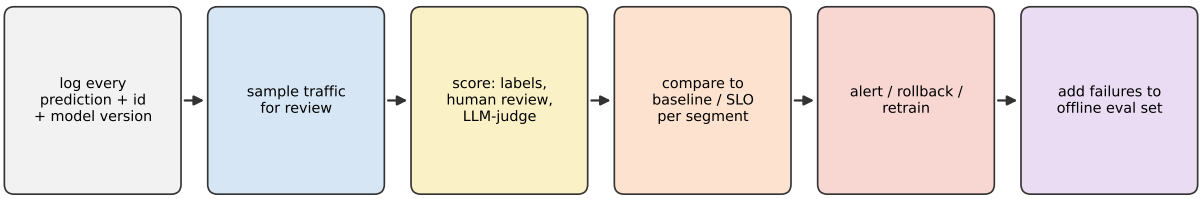


Figure 10.2: Continuous evaluation in production.

10.3 The post-deployment evaluation loop

1. **Log** inputs (or hashes), outputs, model and prompt version, latency, cost, user/segment IDs (respecting privacy).
2. **Sample** traffic for review – random, plus edge cases and low-confidence predictions.
3. **Score** with labels when available, human reviewers, or LLM judges (for text).
4. **Compare** with the offline baseline, the SLO and the previous version – **per segment**, not only overall.
5. **Act**: alert, roll back, retrain, or fix data pipelines.
6. **Learn**: every failure becomes a new test case in the offline evaluation set.

10.4 Online evaluation methods

Method	Use
Shadow evaluation	New model scores live traffic silently; compare its outputs and (later) its accuracy with the champion's
Canary analysis	Compare error rate, latency and key KPIs of canary vs baseline before increasing traffic
A/B / interleaving	Measure the causal effect on business metrics (Chapter 7); interleaving mixes two rankers' results in one list – efficient for search and recommendations
Backtesting	Replay recent production data through old and new models
Feedback signals	Explicit (ratings, thumbs) and implicit (edits, retries, abandonment)

10.5 Evaluating LLM and RAG apps in production

- **Online LLM-as-judge**: automatically score a sample of live traces (e.g. 5%) for faithfulness to retrieved context, answer relevance, policy compliance and tone. No reference answer is needed for these.
- **User feedback**: thumbs up/down, "copy" and "regenerate" clicks, escalations to a human agent.
- **Annotation queues**: experts label flagged or sampled conversations; disagreements refine the rubric.
- **Agent metrics**: task success, steps and tool calls per task, tool error rate, cost per completed task.
- **Calibrate the judge**: check agreement between LLM-judge and human labels regularly.
- **Close the loop**: bad production traces → golden dataset → CI regression tests for the next

prompt or model change.

10.6 What "good" looks like: a scorecard

Dimension	Example metric	Example target
Quality	F1 on labelled weekly sample / LLM-judge pass rate	$\geq$ offline value $-$ 2 points
Fairness	Recall gap between groups	≤ 0.05
Reliability	Success rate, p95 latency	99.9%, $<$ 300 ms
Data health	PSI of top-10 features	all < 0.2
Cost	Cost per 1,000 predictions / per resolved chat	within budget
Business	Fraud loss avoided, conversion, CSAT	agreed with the business owner
Safety	Guardrail violations, escaped PII	0 critical

CHAPTER 11

AIOps: AI for IT Operations

11.1 What is AIOps?

The idea in plain words

AIOps (Artificial Intelligence for IT Operations) applies machine learning and analytics to operational data – metrics, logs, traces, alerts, tickets, deployments – to **detect problems earlier, cut alert noise, find root causes faster and automate fixes**. The term was coined by Gartner around 2016–17; Gartner now covers the market as "event intelligence solutions".

Remember

Don't mix them up: **MLOps** = operating ML models. **LLMOps** = operating LLM applications. **AIOps** = using AI to operate IT systems (which may include your ML services). An MLOps platform can be *monitored by* an AIOps platform.

Real-life analogy

A modern hospital monitor doesn't just beep when one number crosses a line; it knows what's normal for this patient at this time of day, notices when several signals change together, and suggests the likely cause.

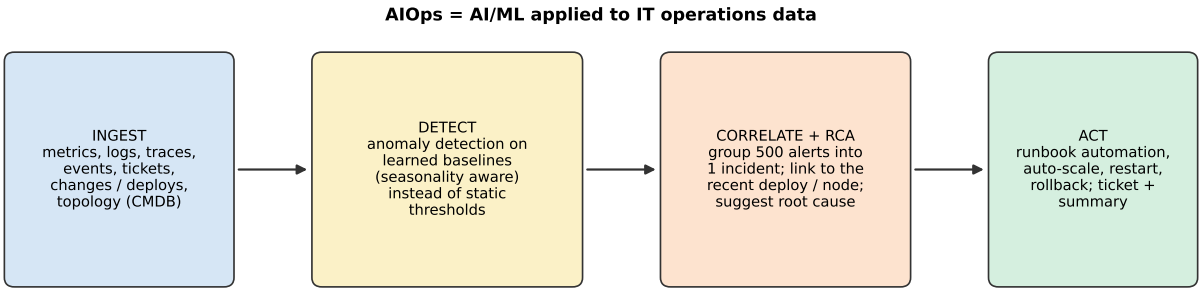


Figure 11.1: The AIOps loop: ingest, detect, correlate, act.

11.2 Core capabilities

11.2.1 1. Anomaly detection

Instead of fixed thresholds ("alert if CPU > 80%"), learn what's **normal** – including daily and weekly patterns – and alert on unusual deviations.

Key equations

z-score on residuals: $z_t = \dfrac{y_t - \hat{y}_t}{\sigma}$, alert if $|z_t| > k$ (e.g. 3–4), where $\hat{y}_t$ = expected value (seasonal baseline, moving average or forecast).

EWMA baseline: $\hat{y}_t = \alpha\, y_{t-1} + (1-\alpha)\, \hat{y}_{t-1}$ (recent points weigh more).

Other methods: seasonal decomposition (STL), forecasting models (Prophet, ARIMA, LSTM), Isolation Forest for multivariate metrics, log-pattern clustering for new error types.

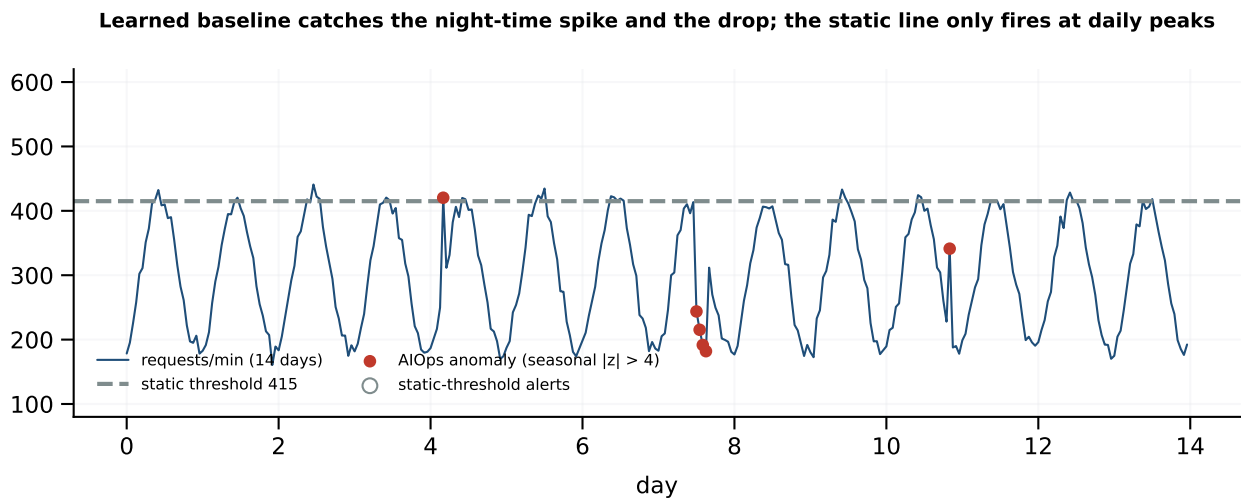


Figure 11.2: Real computation on 14 days of a simulated metric: a seasonal baseline with $|z| > 4$ catches a night-time spike and a sudden drop that a static threshold misses – while the static threshold fires on normal daily peaks.

11.2.2 2. Event correlation and noise reduction

One failing database can trigger hundreds of alerts across services. AIOps groups alerts by **time, topology** (what depends on what) and **text similarity** into a single incident – teams commonly report large reductions in alert volume.

11.2.3 3. Root-cause analysis (RCA)

Rank likely causes using the dependency graph, which component's anomaly started first, and **change correlation** ("errors began 4 minutes after deployment v2.3"). Most incidents follow a change.

11.2.4 4. Automated remediation

Runbook automation: restart a pod, scale out, clear a cache, fail over, roll back a deployment – with human approval for risky actions (the same human-in-the-loop idea as for AI agents).

11.2.5 5. Prediction and capacity planning

Forecast disk, traffic or GPU demand to act before limits are hit; predict hardware failures.

11.2.6 6. GenAI in operations

LLMs summarise incidents, search past tickets and runbooks (RAG), draft post-mortems and suggest queries or fixes – always reviewed by an engineer.

11.3 Measuring operations

Key equations

MTTD = mean time to detect **MTTR** = mean time to resolve/recover

MTBF = mean time between failures **Availability** $\approx \dfrac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$

e.g. a failure every 500 hours, 1 hour to recover $\Rightarrow$ availability $\approx 500/501 = 99.8\%$.

Other useful measures: alert-to-incident ratio (noise), share of incidents auto-resolved, change failure rate, and the DORA metrics (deployment frequency, lead time, change failure rate, time to restore).

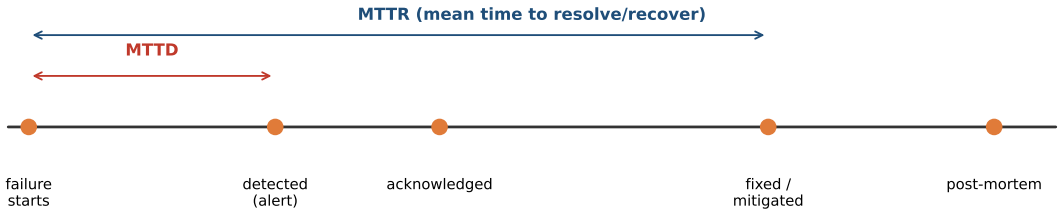


Figure 11.3: Incident timeline.

11.4 AIOps for ML systems

- Feed model-serving metrics (latency, errors, drift scores) into the same observability platform as everything else.
- Correlate a model-quality drop with upstream data-pipeline incidents or schema changes.
- Auto-remediate safely: switch the `@champion` alias back, scale GPU pods, pause a failing pipeline.
- Tools: Dynatrace, Datadog, Splunk ITSI, PagerDuty, BigPanda, ServiceNow ITOM, Moogsoft; cloud-native options in Azure Monitor, CloudWatch and Google Cloud operations.

Watch out

AIOps is only as good as its data: inconsistent tags, missing topology and noisy alerts limit it. Start with clean observability, a few high-value use cases (noise reduction, change correlation) and human review before automation.

CHAPTER 12

Security, Governance, Responsible AI & Explainability

12.1 Security in MLOps

The idea in plain words

An ML system has all the usual software risks *plus* new ones: attackers can poison the data, steal the model through its API, fool it with crafted inputs, or – for LLMs – hijack it with prompts. Security has to cover data, training, artefacts, serving and agents.

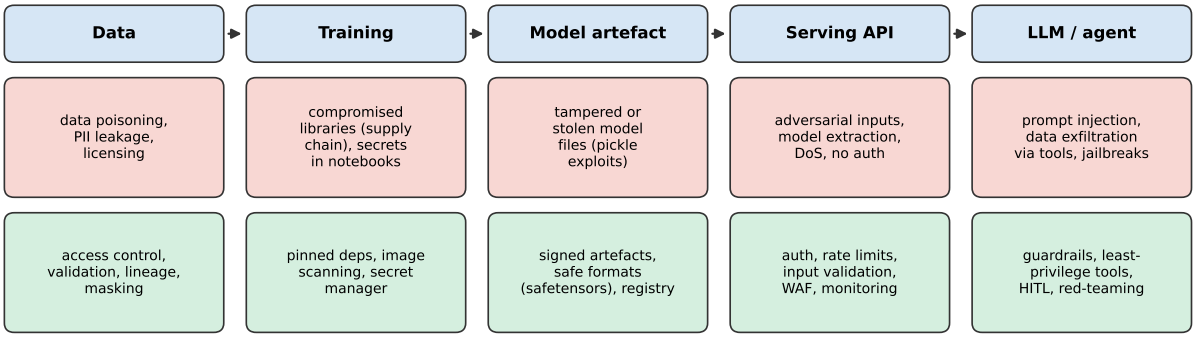


Figure 12.1: Threats (red) and controls (green) across the ML lifecycle.

Threat	Description
Data poisoning	Malicious or corrupted training data plants wrong behaviour or a backdoor
Adversarial examples	Small crafted input changes flip the prediction
Model extraction	Many API queries used to copy the model
Membership / model inversion	Learning whether a person was in the training data, or reconstructing data
Supply chain	Malicious packages, pretrained models or pickle files that run code when loaded
Prompt injection	Instructions in user input or retrieved content override the system's rules
Secrets & PII leakage	Keys in notebooks/images; personal data in logs, prompts or outputs

Controls: identity and RBAC (least privilege), private networking, encryption at rest and in transit, secret managers (Key Vault, Secrets Manager), signed and scanned images and model artefacts, safe model formats (e.g. safetensors), input validation and rate limiting, PII masking, audit logs, red-teaming. Frameworks: OWASP Top 10 for LLM applications, MITRE ATLAS.

12.2 Governance

- **Model inventory**: every model in production, its owner, purpose, risk level and version.
- **Lineage and audit trail**: data → code → run → model version → deployment → who approved it and when.

- **Approval gates**: risk-based sign-off (business, risk/compliance, technical) before promotion.
- **Documentation**: **model cards** (intended use, data, metrics, limits, fairness results) and **datasheets for datasets**.
- **Regulation and standards**: data protection laws (GDPR, India's DPDP Act), risk-based AI regulation such as the EU AI Act, and the NIST AI Risk Management Framework and ISO/IEC 42001 as management frameworks. Check what applies to your use case with your compliance team.
- **Retention and deletion**: honour data-deletion requests, including in training sets and logs.

12.3 Responsible AI

Principles commonly used across companies (e.g. Microsoft's): **fairness, reliability & safety, privacy & security, inclusiveness, transparency, accountability**. In MLOps they become *checks in the pipeline*, not just slogans.

12.3.1 Fairness metrics

Key equations

Demographic parity: $P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b)$ (same selection rate)

Disparate impact ratio: $\dfrac{P(\hat{Y} = 1 \mid \text{unprivileged})}{P(\hat{Y} = 1 \mid \text{privileged})}$; below 0.8 is a common red flag ("four-fifths rule").

Equal opportunity: equal true positive rates across groups. **Equalised odds:** equal TPR *and* FPR.

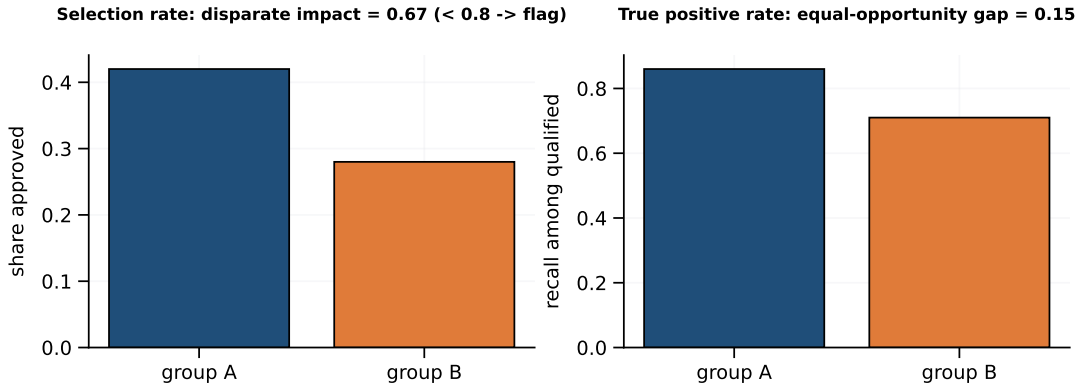


Figure 12.2: Illustrative fairness check: selection-rate ratio 0.67 and a recall gap of 0.15 would both be flagged.

- Fairness metrics can conflict; choose with domain and legal experts.
- Mitigation: better/balanced data, reweighting, fairness-constrained training, group-aware thresholds, human review. Tools: Fairlearn, AIF360, SageMaker Clarify, Azure Responsible AI dashboard.
- Removing the sensitive column is not enough – proxies (postcode, name) can carry the same information.

12.4 Model explainability

The idea in plain words

Explainability answers "why did the model decide this?" – globally (what drives the model overall) and locally (why this one prediction). It builds trust, helps debugging, and is often required for decisions such as credit.

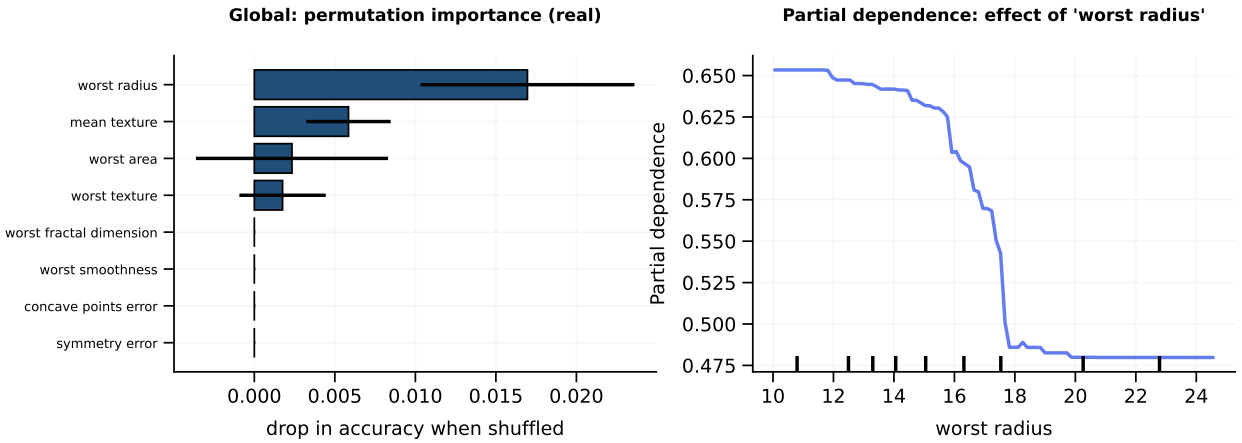


Figure 12.3: Real global explanations for a random forest: permutation importance and a partial dependence plot.

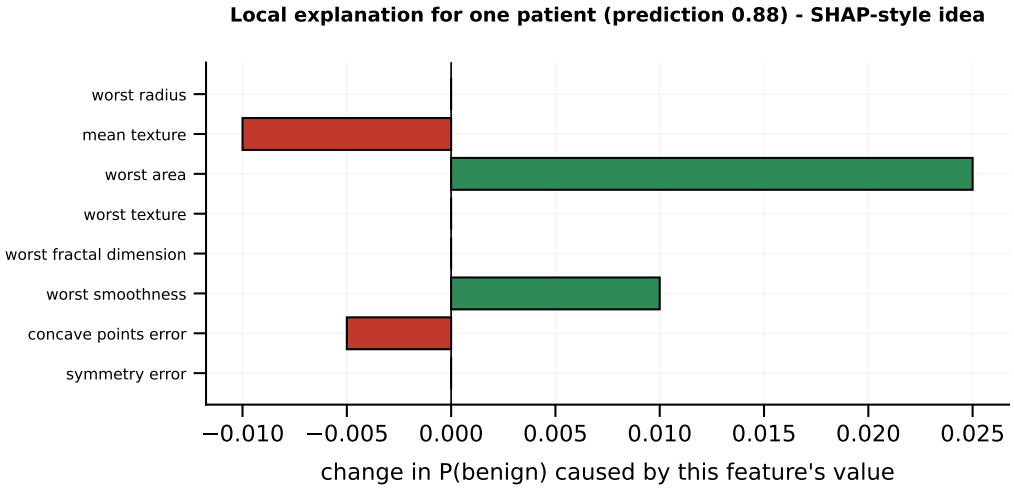


Figure 12.4: Real local explanation for one prediction: which feature values pushed it up or down (a simple replacement-based version of the SHAP idea).

Key equations

SHAP: $f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$ – each feature's contribution ϕ_j (its Shapley value from game theory) adds to the baseline ϕ_0 (the average prediction) to give exactly this prediction.

Permutation importance: $\text{importance}_j = \text{score(original)} - \text{score(feature } j \text{ shuffled)}$.

Method	Scope	Notes
Coefficients / trees	Global	Interpretable-by-design models
Permutation importance	Global	Model-agnostic; misleading with correlated features
Partial dependence / ICE	Global/local	Average effect of one feature on the prediction
SHAP	Local + global	Consistent additive attributions; TreeSHAP is fast for tree models
LIME	Local	Fits a simple surrogate model around one prediction
Counterfactuals	Local	"Approved if income were 10% higher" – actionable
Grad-CAM, saliency	Local	Images (see the DL guide)
Citations, traces	Local	LLM/RAG: show sources and steps

CHAPTER 13

MLOps Best Practices & Design Patterns

13.1 Best practices checklist

Area	Practice
Versioning	Git for code; data versions/hashes; registry for models; pinned environments; prompts as artefacts
Reproducibility	Seeds, logged params and environment, containerised training, one command to rebuild a model
Pipelines	Automate train → evaluate → register; idempotent, parameterised, cached steps
Testing	Unit + data + model + behavioural + fairness + latency tests as CI gates
Deployment	Containers, registry aliases, canary/shadow, instant rollback, infrastructure as code (Terraform, Bicep)
Monitoring	Service SLOs, data and prediction drift, delayed-label performance, business KPIs, per-segment views
Retraining	Triggered or scheduled CT with champion–challenger gates; never auto-deploy without validation
Security	Least privilege, secret manager, scanned images, PII controls, audit logs
Responsible AI	Fairness and explainability checks, model cards, human oversight for high-risk decisions
Cost	Autoscaling, right-sized hardware, spot/batch, LLM routing and caching, budgets and alerts
People	Clear owners for each model, runbooks, on-call, blameless post-mortems

13.2 Design patterns

Pattern	Idea
Model-as-a-service	Model behind a versioned API (FastAPI / TF Serving / managed endpoint)
Batch scoring	Scheduled job writes predictions to a table the business reads
Embedded model	Model shipped inside the application or device (edge)
Feature store	One definition of features for training and serving
Champion–challenger	New model must beat the current one before promotion
Shadow deployment	Test on live traffic without affecting users
Ensemble / cascade	Cheap model first, expensive model for hard cases (also for LLM routing)
Fallback / circuit breaker	If the model or provider fails, return a safe default or rule-based answer
Human-in-the-loop	Low confidence or high risk → human review; their labels feed retraining
Prediction logging + label join	Every prediction gets an ID so outcomes can be joined later
Two-phase predictions	Fast simple model on device, complex model in the cloud
Workflow pipeline	Each step a container in an orchestrated DAG

13.3 Anti-patterns to avoid

Notebook-only production code; manual copy of model files; preprocessing duplicated between training and serving; “latest” image tags; monitoring only CPU and uptime; retraining and auto-deploying without comparison; evaluating only on overall averages; secrets in code or prompts; no rollback plan.

CHAPTER 14

Cheat Sheet & Interview Revision

14.1 Formula sheet

Concept	Formula
HPA replicas	$\lceil$replicas $\times$ current metric / target metric$\rceil$
Little's law	in-flight requests = throughput $\times$ latency
INT8 quantisation	$q = \text{round}(w/s)$, $s = \max\|w\|/127$
Error budget	$1 -$ SLO (e.g. 0.1% of requests for 99.9%)
PSI	$\sum(a_i - e_i)\ln(a_i/e_i)$; < 0.1 ok, > 0.25 significant
KS statistic	$\max\|F_{\text{ref}} - F_{\text{cur}}\|$
Chi-square	$\sum(O - E)^2/E$
A/B z-test	$(\hat{p}_B - \hat{p}_A)/\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}$
LLM cost / request	$(T_{\text{in}}P_{\text{in}} + T_{\text{out}}P_{\text{out}})/10^6$
LLM latency	TTFT + output tokens $\times$ TPOT
Availability	MTBF / (MTBF + MTTR)
Disparate impact	selection rate (unprivileged) / selection rate (privileged), flag < 0.8
SHAP	$f(x) = \phi_0 + \sum_j \phi_j$

14.2 Interview quick revision

What is MLOps? Practices and tools to deploy, monitor and maintain ML models reliably: versioning, automation (CI/CD/CT), testing, monitoring, governance.

MLOps vs DevOps? MLOps also versions data and models, tests data and model quality, adds continuous training, and monitors drift because ML fails silently.

What does MLflow track? Parameters, metrics, artifacts, tags, code version and models per run; the registry adds versions, aliases, tags and lineage.

Stages vs aliases? Stages (Staging/Production) are the older, deprecated MLflow approach; aliases like `@champion` are movable pointers – promote or roll back by reassigning.

Batch vs real-time inference? Batch scores many rows on a schedule (cheap, latency-tolerant); real-time answers each request in milliseconds via an API.

Blue-green vs canary vs shadow? Blue-green switches all traffic between two environments; canary sends a small, growing share to the new version; shadow sends a copy whose answers aren't used.

Canary vs A/B test? Canary limits rollout risk; A/B measures which version is better on a business metric with statistics.

Why FastAPI for ML? Fast, type-hinted, automatic Pydantic validation and Swagger docs; load the model once at startup.

What is TensorFlow Serving? A production server for SavedModels with REST (8501) and gRPC (8500), versioning and batching.

Docker image vs container? Image = packaged read-only template; container = running instance of it.

Why Kubernetes? Runs and scales many containers: self-healing, load balancing, rolling updates, autoscaling (HPA), GPU scheduling.

Name model optimisation techniques. Quantisation, pruning, distillation, ONNX/TensorRT compilation, batching, caching.

Latency vs throughput? Latency = time per request; throughput = requests per unit time.

What is CT? Continuous training: retraining triggered by schedule, drift or new data, then evaluation and gated promotion.

What should ML tests cover? Code (unit), data (schema, ranges, nulls, leakage), model (thresholds, slices, behavioural, fairness), non-functional (latency).

Data drift vs concept drift? Data drift: $P(X)$ changes. Concept drift: $P(Y|X)$ changes – the same input now means a different outcome.

How do you detect drift? Compare reference and current windows: PSI, KS, chi-square, JS, Wasserstein; dataset-level rules; performance once labels arrive.

What is PSI and its thresholds? $\sum(a-e)\ln(a/e)$ over bins; < 0.1 stable, 0.1–0.25 moderate, > 0.25 significant.

Retraining strategies? Scheduled, trigger-based, continuous/online; full, sliding window or incremental data; always champion–challenger validation.

What is pipeline orchestration? Running DAGs of tasks with dependencies, schedules, retries and logging (Airflow, Kubeflow, cloud pipelines).

SageMaker vs Azure ML vs Vertex AI? Same lifecycle concepts on AWS, Azure and Google Cloud: managed training, tracking, registry, pipelines, endpoints, monitoring.

What is LLMOps? Operating LLM apps: prompt/model/index versioning, LLM evaluation, guardrails, gateways, cost and latency control, tracing.

How do you cut LLM costs? Smaller models and routing, shorter prompts, fewer chunks, prompt and response caching, output limits, batch APIs, fewer agent steps, self-hosting at scale.

What is TTFT? Time to first token – the main perceived latency for streaming LLM responses.

How do you evaluate a model after deployment? Log predictions with IDs, use proxies (drift, confidence) until labels arrive, join labels, human/LLM-judge review on samples, compare per segment, shadow/canary/A/B, feed failures back into tests.

What is AIOps? AI for IT operations: anomaly detection, event correlation, root-cause analysis and automated remediation on operational data.

MTTD vs MTTR? Mean time to detect vs mean time to resolve/recover.

Fairness metrics? Demographic parity, disparate impact (four-fifths rule), equal opportunity, equalised odds.

SHAP vs LIME? SHAP: Shapley-value additive contributions, consistent, local and global. LIME: local surrogate model around one prediction.

Key ML security risks? Data poisoning, adversarial examples, model extraction, supply-chain attacks, prompt injection, secret/PII leakage.

Further Reading

1. D. Sculley et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS 2015.
2. Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning (maturity levels 0–2).
3. C. Huyen. *Designing Machine Learning Systems*. O'Reilly, 2022.
4. E. Breck et al. The ML Test Score: A Rubric for ML Production Readiness. 2017.
5. M. T. Ribeiro et al. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. ACL 2020.
6. Google. *Site Reliability Engineering* (golden signals, SLOs, error budgets).
7. MLflow documentation: tracking, models, model registry (aliases and tags).
8. Evidently AI documentation: data drift methods and defaults.
9. S. Lundberg, S.-I. Lee. A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS 2017.
10. OWASP Top 10 for LLM Applications; MITRE ATLAS; NIST AI Risk Management Framework.
11. AWS SageMaker, Azure Machine Learning and Google Vertex AI documentation.